SHARING KNOWLEDGE.
TOGETHER.

WORKPLACE NINJA SUMMIT

SUMMIT

SUMMIT

SUMMIT

SUMMIT

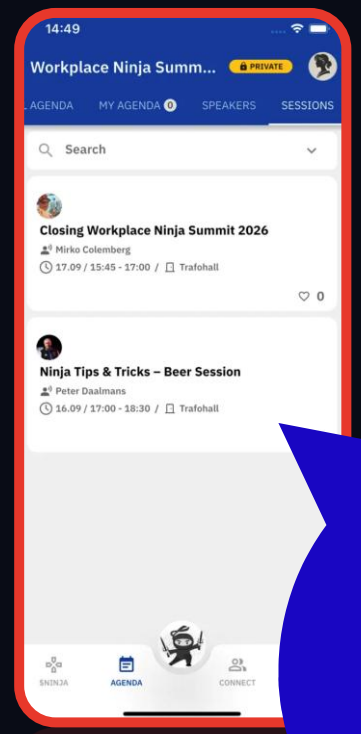
SUMMIT



Check in and rate this session in the Workplace Ninja Summit 2026 app

GET IT ON Google Play Store

Download on the App Store



We appreciate your feedback!

WORKPLACE NINJA SUMMIT

Hertz
room check-in



WORKPLACE NINJA SUMMIT

Windows 365 License Management

the Graph API edition



Thank you to our sponsors!

WORKPLACE NINJA SUMMIT

PLATINUM SPONSORS

glueck kanja

PATCH MY PC

Recast

2Pint

Microsoft

GOLD SPONSORS

control up

Admin By Request
ZERO TRUST PLATFORM

eido

THREATLOCKER®
ZERO TRUST PLATFORM

inforcer

baseVISION
Trusted Swiss Cybersecurity

robopack

EPIC FUSION
BRING IT ALL TOGETHER

nerdio

SILVER SPONSORS

systemcenterdudes

Apento

water
IT Security & Defense

ugurlabs

Welcome on stage

WORKPLACE NINJA SUMMIT



Session

1. Windows 365 Cloud PC License Management automation
2. Graph API | PowerShell | YAML
3. Azure DevOps Pipeline approval
4. Teams notifications

Speaker

Esther Barthel

Senior Solutions Architect @ cognition IT

#wpninjas

About Esther

WORKPLACE NINJA SUMMIT

Focus

Senior Solutions Architect





From

The Netherlands



Blog

https://www.cloud-devops-ninja/

Certifications





Hobbies





KAMADOJOE®

MARVEL STUDIOS

Contact

github.com/cloud-devops-ninja

linkedin.com/in/ebarthel/

WORKPLACE NINJA SUMMIT

End User Computing




plex





AVD



W365 CloudPC



W365 CloudPC



W365 CloudPC

WORKPLACE NINJA SUMMIT

Infrastructure-as-Code

WORKPLACE NINJA SUMMIT

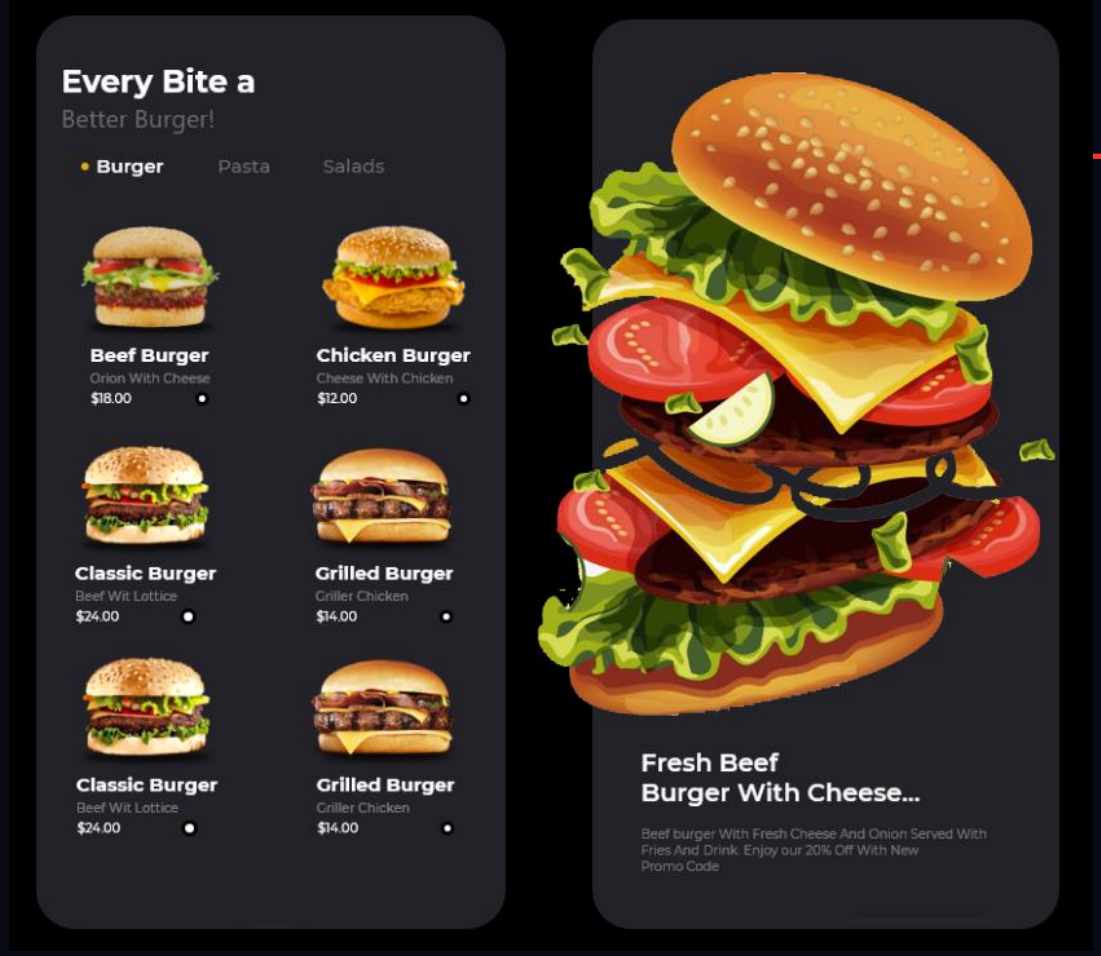
*a modern approach to **managing & provisioning** IT **infrastructure** through the use of **code**, rather than traditional manual processes.*



W365 CloudPC (Config-as-Code)

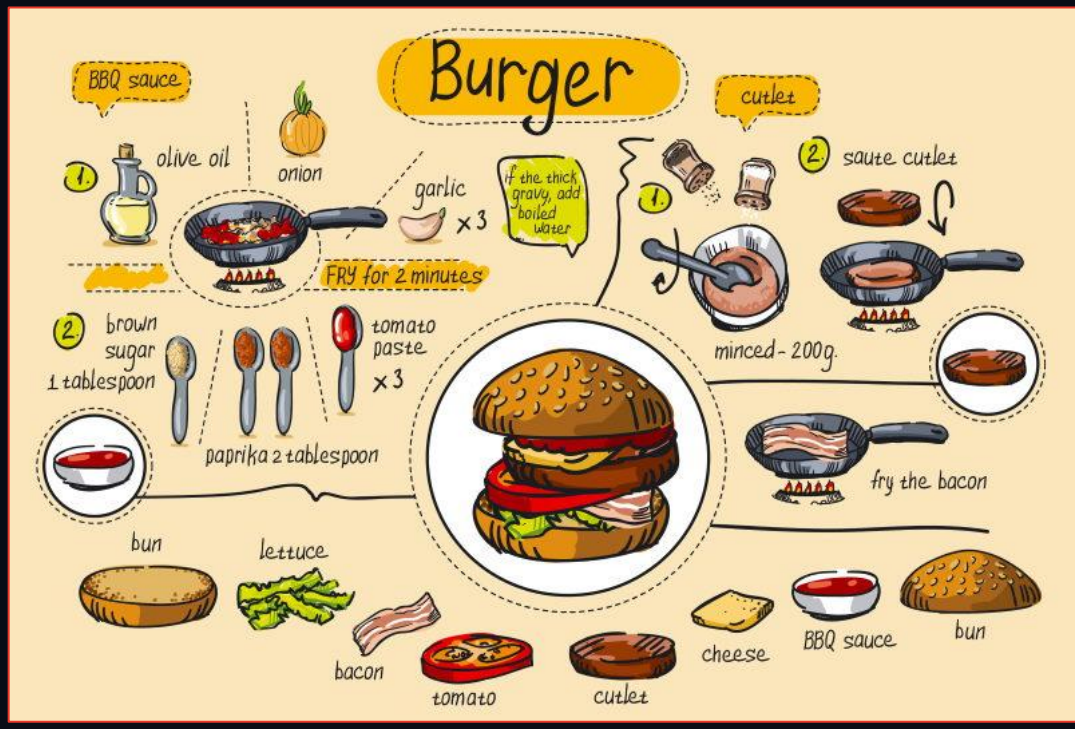


W365 CloudPC (Config-as-Code)



Declarative code
(templates)





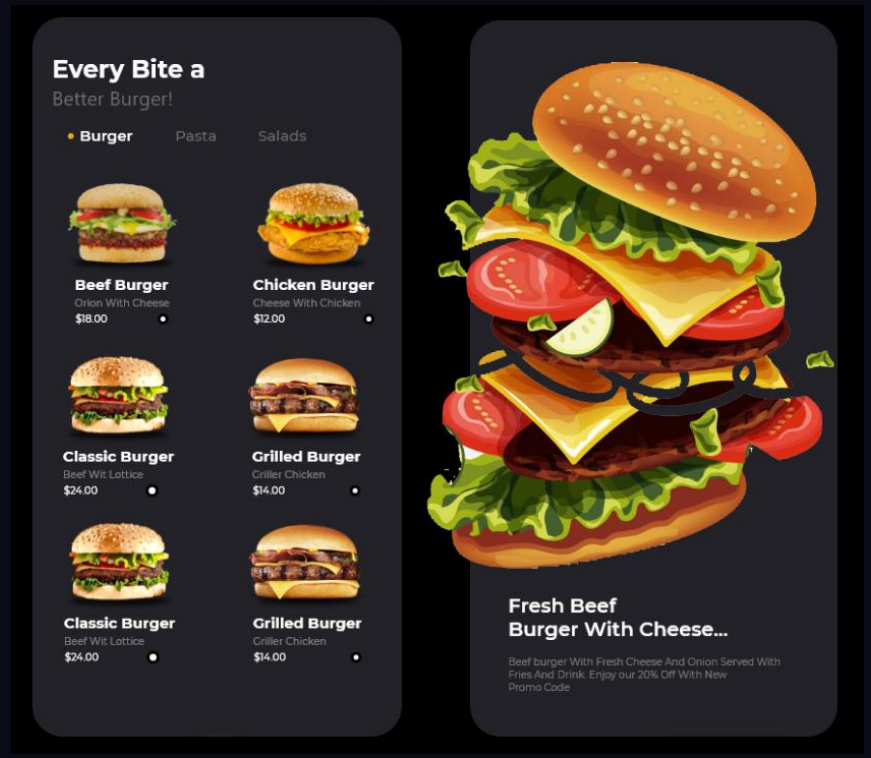
Imperative code
(scripts)



WORKPLACE NINJA SUMMIT

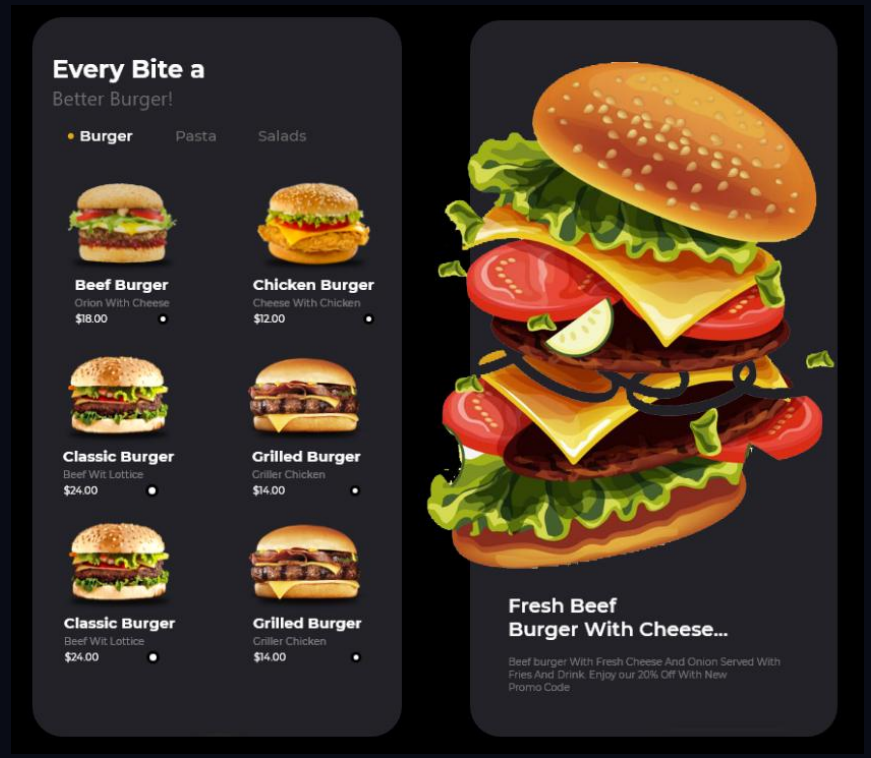
Getting started with the Graph API

Microsoft Graph API



Declarative code (templates)

Microsoft Graph



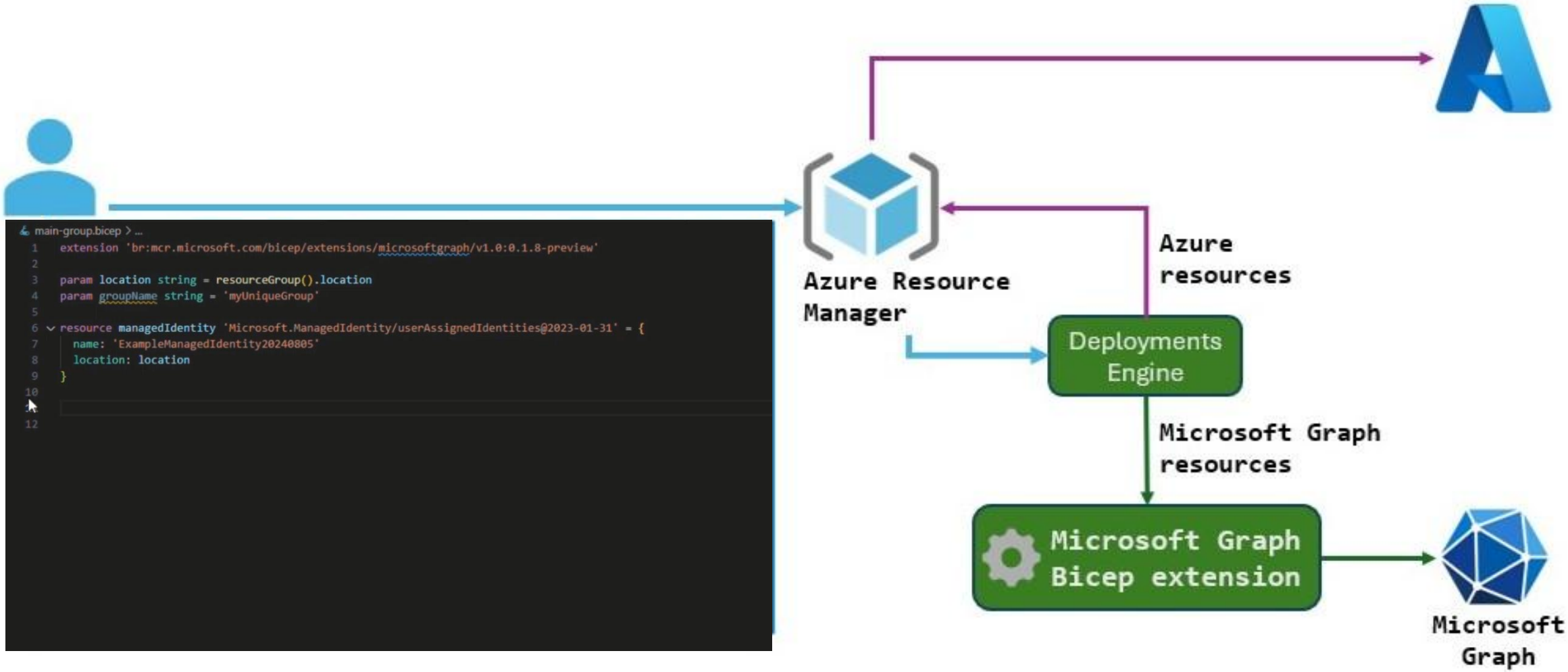
Declarative code (templates)

main-group.bicep > ...

```bicep
extension 'br:mcr.microsoft.com/bicep/extensions/microsoftgraph/v1.0:0.1.8-preview'

param location string = resourceGroup().location
param groupName string = 'myUniqueGroup'

resource managedIdentity 'Microsoft.ManagedIdentity/userAssignedIdentities@2023-01-31' = {
  name: 'ExampleManagedIdentity20240805'
  location: location
}
```



Version

Microsoft Graph Bicep v1.0

Find by title

Bicep templates for Microsoft Graph documentation

Overview

Bicep templates for Microsoft Graph resources

What's new

Microsoft Graph Bicep v1.0 reference

Overview

Applications

Federated identity credentials

App role assigned to

Groups

Oauth2 permission grants

Service principals

Users

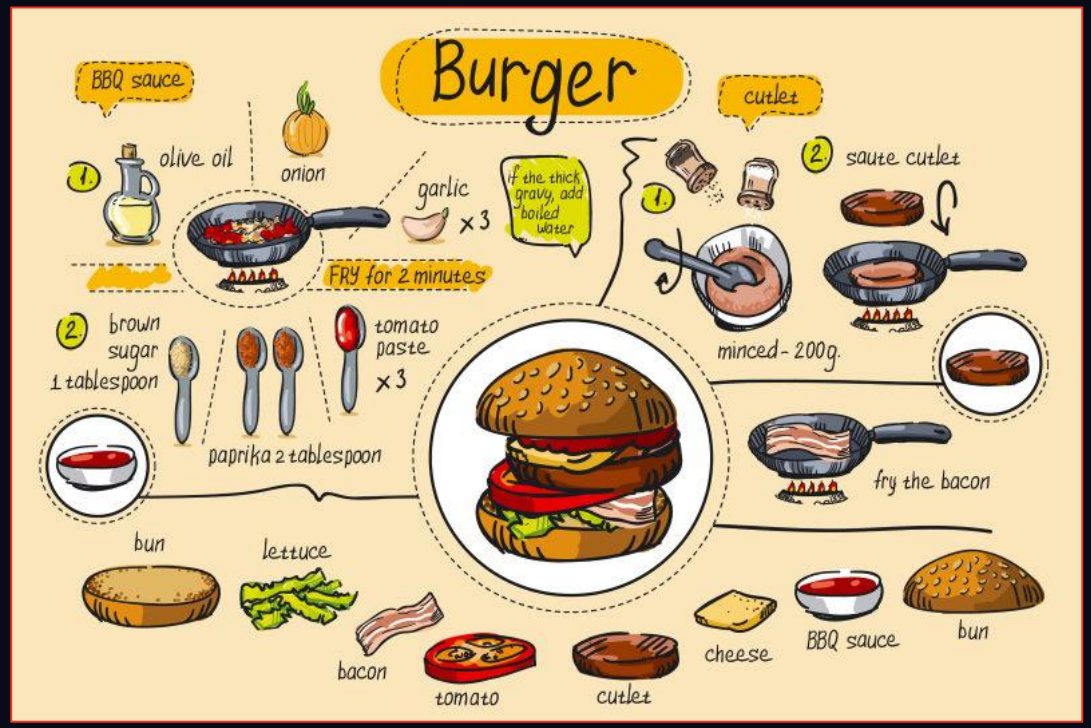
Microsoft Graph Bicep v1.0 resource reference overview

This article gives you an overview of the Microsoft Graph resource types you can use in Bicep files, for the `v1.0` endpoint.

Supported resources

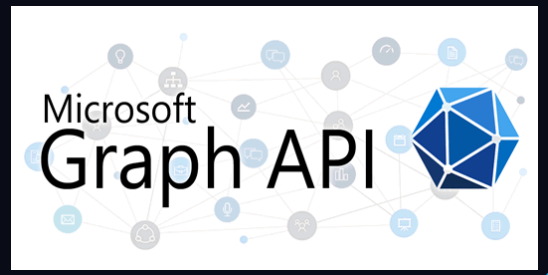
Service	Microsoft Graph Bicep resource type
Applications	Microsoft.Graph/applications
App role assignments	Microsoft.Graph/appRoleAssignedTo
Federated identity credentials	Microsoft.Graph/applications/federatedIdentityCredentials
Groups	Microsoft.Graph/groups
OAuth2 permission grants (delegated permission grants)	Microsoft.Graph/oauth2PermissionGrants
Service principals	Microsoft.Graph/servicePrincipals
Users	Microsoft.Graph/users

Microsoft Graph API



Imperative code (scripts)


Coding skills loading



Rest(full) API



PowerShell SDK



Microsoft Learn: Microsoft Graph REST API v1.0 endpoint reference

Microsoft Learn: Get started w/ Microsoft Graph PowerShell SDK

Graph Explorer

Sample queries | Resources

Search resources

Switch to beta Off

Resources available

- groups (8)
  - GET
  - POST
  - {group-id} (46)
  - $count (1)

GET v1.0 https://graph.microsoft.com/v1.0/groups?$filter=startswith(displayName, 'grp-sec')&$select=id,displayName,description Run query

Request body | Request headers | Modify permissions | Access token

OK - 200 - 333 ms

Response preview | Response headers | Code snippets | Toolkit component | Adaptive cards


```
{
    "@odata.context": "https://graph.microsoft.com/v1.0/$metadata#groups(id,displayName,description)",
    "value": [
        {
            "id": "[redacted]",
            "displayName": "grp-sec-AVDUsers",
            "description": "AVD Users group"
        }
    ]
}
```


Response preview | Response headers | Code snippets | Toolkit component | Adaptive cards

C# | CLI | Go | Java | JavaScript | PHP | PowerShell | Python

```
# Leverage the power of our client libraries. Download the PowerShell client library here https://aka.ms/pshellsdk
# To read more about our SDKs, go to the documentation page at https://aka.ms/pshellsdkdocs
Import-Module Microsoft.Graph.Groups

Get-MgGroup -Filter "startswith(displayName, 'grp-sec')" -Property "id,displayName,description"
```

WORKPLACE NINJA SUMMIT

Graph API reference docs

Use the Microsoft Graph API

Microsoft Graph is a RESTful web API that enables you to access Microsoft Cloud service resources. After you register your app and get authentication tokens for a user or service, you can make requests to the Microsoft Graph API.

Call a REST API method

To read from or write to a resource such as a user or an email message, you construct a request that looks like the following:

HTTP Copy

```
{HTTP method} https://graph.microsoft.com/{version}/{resource}?{query-parameters}
```

4. Request an access token

In the OAuth 2.0 client credentials grant flow, you use the application ID and client secret values that you saved when you registered your app to request an access token directly from the Microsoft identity platform `/token` endpoint.

You specify the preconfigured permissions by passing `https://graph.microsoft.com/.default` as the value for the `scope` parameter in the token request.

Send a POST request to the `/token` identity platform endpoint to acquire an access token. In this request, the client uses the client secret.

HTTP cURL

HTTP Copy

request, the client uses the client secret.

HTTP cURL

HTTP Copy

```
// Line breaks are for legibility only.

POST https://login.microsoftonline.com/{tenant}/oauth2/v2.0/token HTTP/1.1
Host: login.microsoftonline.com
Content-Type: application/x-www-form-urlencoded

client_id=535fb089-9ff3-47b6-9bfb-4f1264799865
&scope=https%3A%2F%2Fgraph.microsoft.com%2F.default
&client_secret=qWgdYA....L1qKv5bPX
&grant_type=client_credentials
```

request, the client uses the client secret.

HTTP cURL

HTTP Copy

```
// Line breaks are for legibility only.

POST https://login.microsoftonline.com/{tenant}/oauth2/v2.0/token HTTP/1.1
Host: login.microsoftonline.com
Content-Type: application/x-www-form-urlencoded

client_id=535fb089-9ff3-47b6-9bfb-4f1264799865
&scope=https%3A%2F%2Fgraph.microsoft.com%2F.default
&client_secret=qWgdYA....L1qKv5bPX
&grant_type=client_credentials
```

WORKPLACE NINJA SUMMIT

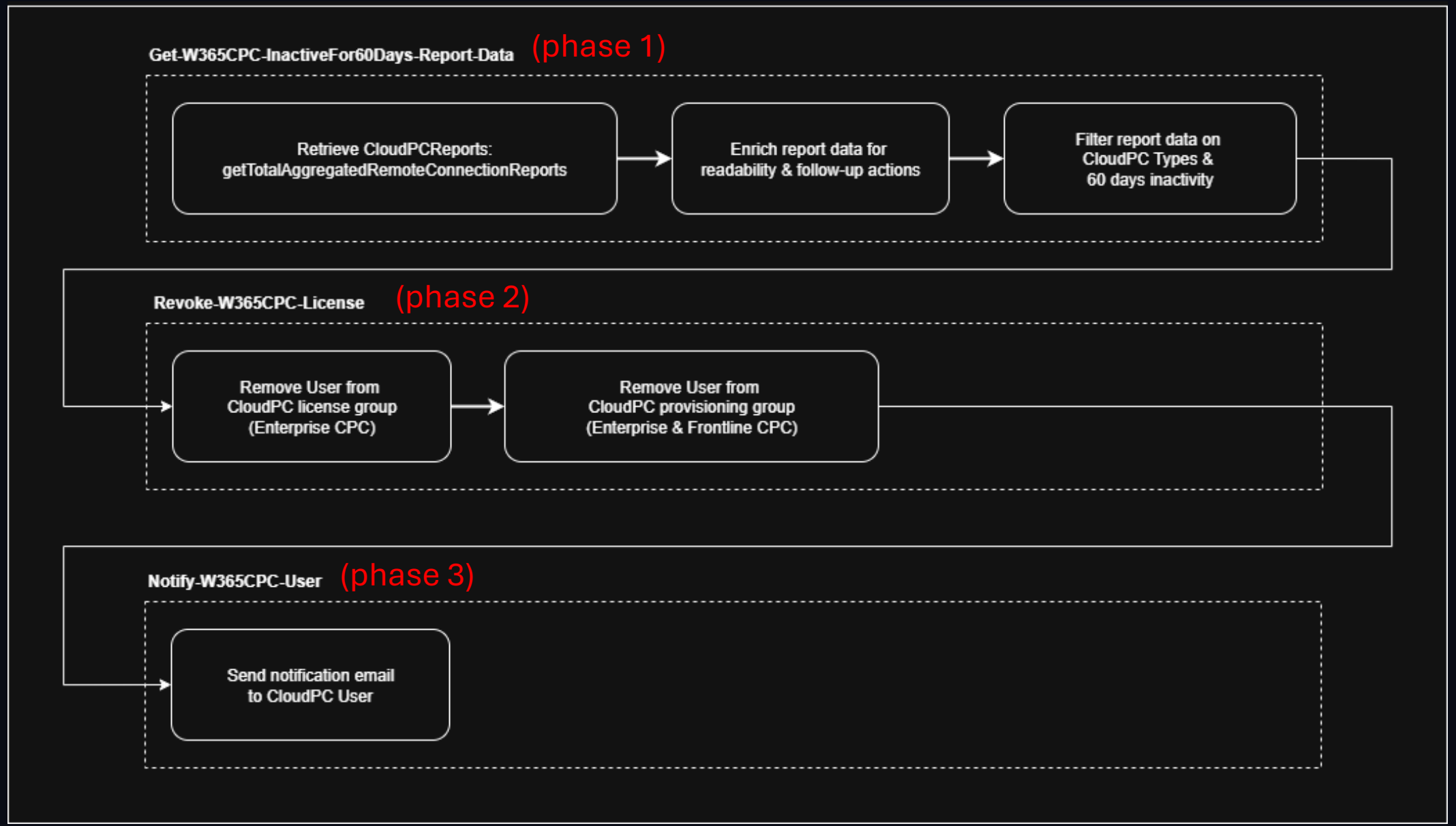
IT Processes

W365 License Management

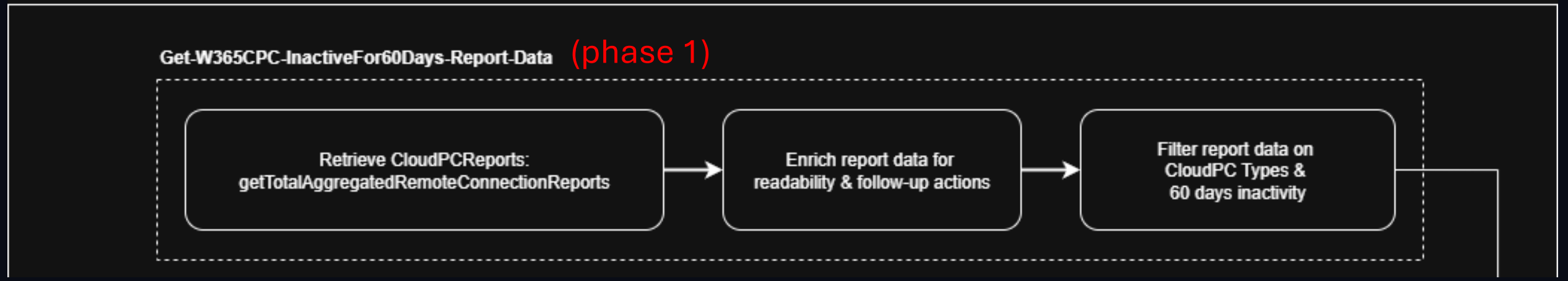
The workflow



W365 CloudPC License Management



W365 CloudPC License Management



W365 CloudPC License Management



HTTP request

HTTP Copy

```
POST /deviceManagement/virtualEndpoint/reports/getTotalAggregatedRemoteConnectionReports
```

Request headers

Expand table

Name	Description
Authorization	Bearer {token}. Required. Learn more about authentication and authorization.
Content-Type	application/json. Required.

Request body

In the request body, supply a JSON representation of the parameters.

The following table shows the parameters that can be used with this action.

Expand table

Parameter	Type	Description
filter	String	OData filter syntax. Supported filters include `and`, `or`, `lt`, `le`, `gt`, `ge`, and `eq`.
select	String collection	OData select syntax. Represents the selected columns of the reports.
search	String	Specifies a string to search
groupBy	String collection	Specify how to group the reports. If used, must have the same contents as select parameter
orderBy	String collection	Specify how to sort the reports.
skip	Int32	Number of records to skip.
top	Int32	The number of top records to return.

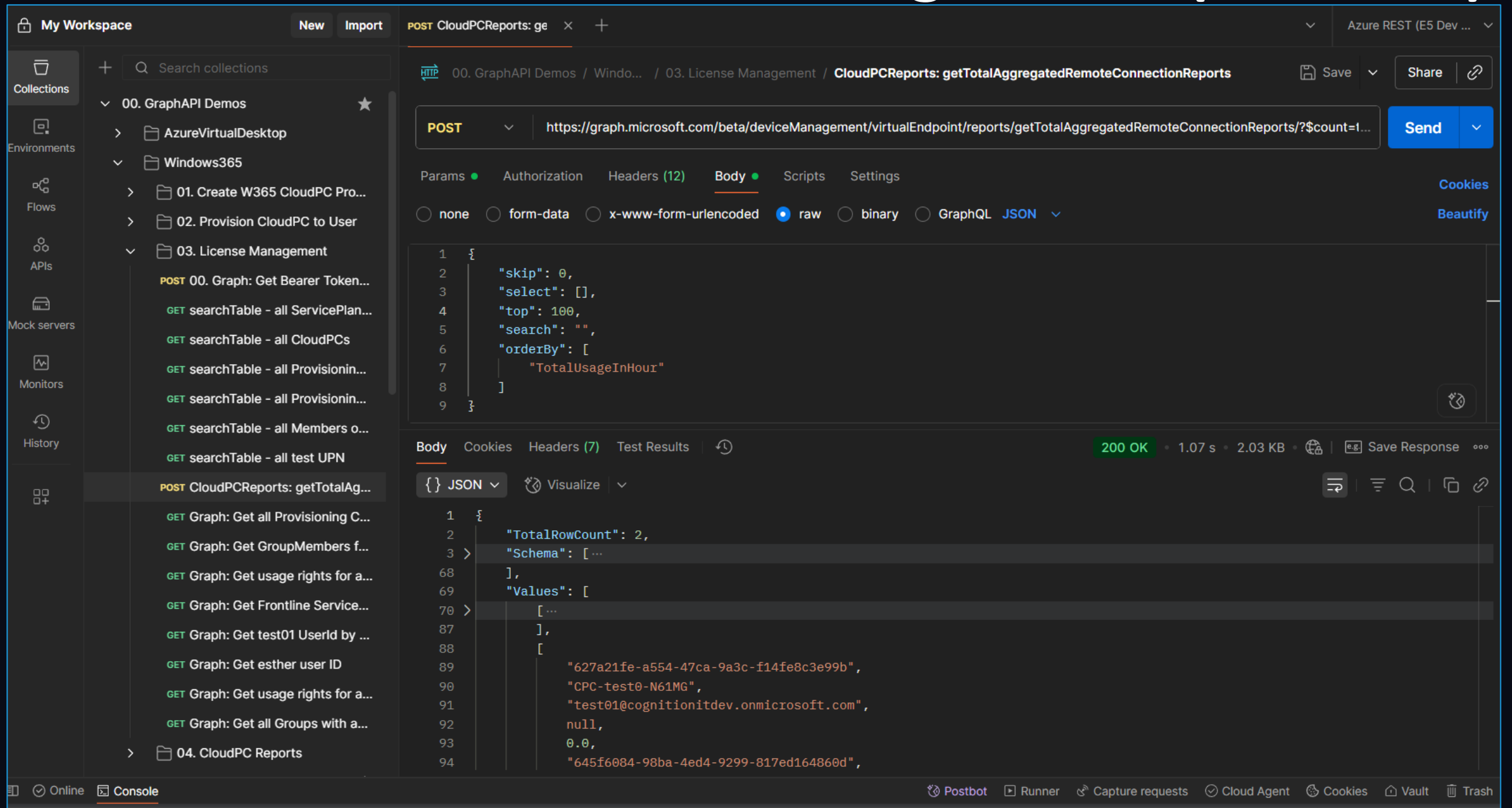
cloudPcReports: getTotalAggregatedRemoteConnectionReports - Microsoft Graph beta | Microsoft Learn

W365 License Management

Postman API testing



W365 CloudPC License Management (Postman)



W365 CloudPC License Management (Postman)



00. GraphAPI Demos / Windo... / 03. License Management / **CloudPCReports: getTotalAggregatedRemoteConnectionReports**

Save | Share

POST https://graph.microsoft.com/beta/deviceManagement/virtualEndpoint/reports/getTotalAggregatedRemoteConnectionReports/?$count=1... **Send**

Params • Authorization Headers (12) Body • Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
{
    "skip": 0,
    "select": [],
    "top": 1,
    "search": "",
    "orderBy": [
        "TotalUsageInHour"
    ]
}
```

Body Cookies Headers (7) Test Results 200 OK • 783 ms • 1.64 KB Save Response

{} JSON Visualize

```
{
    "TotalRowCount": 2,
    "Schema": [...
    ],
    "Values": [...
    ]
}
```

cloudPcReports: getTotalAggregatedRemoteConnectionReports - Microsoft Graph beta | Microsoft Learn

W365 CloudPC License Management (Postman)



00. GraphAPI Demos / Windo... / 03. License Management / **CloudPCReports: getTotalAggregatedRemoteConnectionReports**

Save | Share

POST https://graph.microsoft.com/beta/deviceManagement/virtualEndpoint/reports/getTotalAggregatedRemoteConnectionReports/?$count=1... Send

Params • Authorization Headers (12) Body • Scripts Settings Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL JSON Beautify

```
{
    "skip": 0,
    "select": [],
    "top": 1,
    "search": "",
    "orderBy": [
        "TotalUsageInHour"
    ]
}
```

Body Cookies Headers (7) Test Results 200 OK • 783 ms • 1.64 KB Save Response

{} JSON Visualize

```
{
    "TotalRowCount": 2,
    "Schema": [...
```

ManagedDeviceName	DeviceType	NeverSignedIn	LastActiveTime	CreatedDate	DaysSinceLastSignIn	TotalUsageInHour	UserPrincipalName	IntuneDeviceId	PolicyName	ServicePlanId	CloudPCSize	CloudPcId
TST-test0-B5JM4	Enterprise	TRUE		2025-12-05 12:07		0	test0@cognitionittst.onmicrosoft.com	12345678-9012-3	TST Test Cloud PC	2de9c682-1234-1234-1234-1234567890	Cloud PC Enterprise 4vCPU/16GB/128GE	64379843-123
TST-test1-KP47C	Enterprise	FALSE	6/20/2025 2:37:02 PM	2025-12-05 12:07	56	0	test1@cognitionittst.onmicrosoft.com	12345678-9012-3	TST Test Cloud PC	69dc175c-1234-1234-1234-1234567890	Cloud PC Enterprise 8vCPU/32GB/256GE	cad55617-123
DEV-test0-VEOL3	Enterprise	FALSE	7/16/2025 9:24:11 AM	7/14/2023 9:37:07 AM	30	0	test0@cognitionitdev.onmicrosoft.com	12345678-9012-3	DEV Development Cloud PC	2de9c682-1234-1234-1234-1234567890	Cloud PC Enterprise 4vCPU/16GB/128GE	82b02dbd-123
DEV-test1-ZSSX8	FrontlineDedicated	FALSE	7/16/2025 5:55:31 PM	2025-09-07 08:38	30	0	test1@cognitionitdev.onmicrosoft.com	12345678-9012-3	DEV Development Cloud PC FL	057efbfe-1234-1234-1234-12345678901	Cloud PC Frontline 2vCPU/8GB/128GB	6a4f5c5d-1234
DEV-test2-FAUF8	Enterprise	FALSE	6/30/2025 7:09:21 AM	2024-08-04 06:28	46	0	test2@cognitionitdev.onmicrosoft.com	12345678-9012-3	DEV Development Cloud PC	3efff3fe-1234-1234-1234-123456789012	Cloud PC Enterprise 2vCPU/8GB/128GB	25b0a8c9-123
DEV-test3-D1U44	FrontlineDedicated	TRUE		8/13/2025 8:08:52 AM		0	test3@cognitionitdev.onmicrosoft.com	12345678-9012-3	DEV Development Cloud PC FL	057efbfe-1234-1234-1234-12345678901	Cloud PC Frontline 2vCPU/8GB/128GB	ccecb673-1234
PRD-test1-01387	FrontlineDedicated	TRUE		2025-06-05 07:45		0	test1@cognitionit.com	12345678-9012-3	PRD Production Cloud PC	057efbfe-1234-1234-1234-12345678901	Cloud PC Frontline 2vCPU/8GB/128GB	a65f5965-1234
PRD-test2-0EU22	FrontlineDedicated	FALSE	2025-11-07 12:57	11/14/2024 10:04:12 AM	35	0	test2@cognitionit.com	12345678-9012-3	PRD Production Cloud PC	057efbfe-1234-1234-1234-12345678901	Cloud PC Frontline 2vCPU/8GB/128GB	1d044521-123

W365 CloudPC License Management (raw)



00. GraphAPI Demos / Windo... / 03. License Management / **CloudPCReports: getTotalAggregatedRemoteConnectionReports**

Save | Share

POST https://graph.microsoft.com/beta/deviceManagement/virtualEndpoint/reports/getTotalAggregatedRemoteConnectionReports/?$count=1... Send

Params • Authorization Headers (12) Body • Scripts Settings Cookies

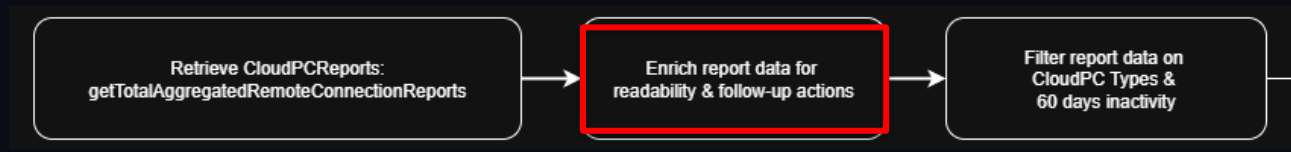
none | form-data | x-www-form-urlencoded | raw | binary | GraphQL JSON Beautify

```
{
    "skip": 0,
    "select": [],
    "top": 1,
    "search": "",
    "orderBy": [
        "TotalUsageInHour"
    ]
}
```

Body Cookies Headers (7) Test Results 200 OK • 783 ms • 1.64 KB Save Response

UserPrincipalName	IntuneDeviceId	PolicyName	ServicePlanId	CloudPCSize	CloudPcId
test0@cognitionittst.onmicrosoft.com	12345678-9012-3	TST Test Cloud PC	2de9c682-1234-1234-1234-1234567890	Cloud PC Enterprise 4vCPU/16GB/128GE	64379843-123
test1@cognitionittst.onmicrosoft.com	12345678-9012-3	TST Test Cloud PC	69dc175c-1234-1234-1234-1234567890	Cloud PC Enterprise 8vCPU/32GB/256GE	cad55617-1234
test0@cognitionitdev.onmicrosoft.com	12345678-9012-3	DEV Development Cloud PC	2de9c682-1234-1234-1234-1234567890	Cloud PC Enterprise 4vCPU/16GB/128GE	82b02dbd-123
test1@cognitionitdev.onmicrosoft.com	12345678-9012-3	DEV Development Cloud PC FL	057efbfe-1234-1234-1234-12345678901	Cloud PC Frontline 2vCPU/8GB/128GB	6a4f5c5d-1234
test2@cognitionitdev.onmicrosoft.com	12345678-9012-3	DEV Development Cloud PC	3efff3fe-1234-1234-1234-123456789012	Cloud PC Enterprise 2vCPU/8GB/128GB	25b0a8c9-1234
test3@cognitionitdev.onmicrosoft.com	12345678-9012-3	DEV Development Cloud PC FL	057efbfe-1234-1234-1234-12345678901	Cloud PC Frontline 2vCPU/8GB/128GB	ccecb673-1234
test1@cognitionit.com	12345678-9012-3	PRD Production Cloud PC	057efbfe-1234-1234-1234-12345678901	Cloud PC Frontline 2vCPU/8GB/128GB	a65f5965-1234
test2@cognitionit.com	12345678-9012-3	PRD Production Cloud PC	057efbfe-1234-1234-1234-12345678901	Cloud PC Frontline 2vCPU/8GB/128GB	1d044521-123

W365 CloudPC License Management (enriched)



00. GraphAPI Demos / Windo... / 03. License Management / **CloudPCReports: getTotalAggregatedRemoteConnectionReports**

Save | Share

POST https://graph.microsoft.com/beta/deviceManagement/virtualEndpoint/reports/getTotalAggregatedRemoteConnectionReports/?$count=1... Send

Params • Authorization Headers (12) Body • Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Beautify

```
{
    "skip": 0,
    "select": [],
    "top": 1,
    "search": "",
    "orderBy": [
        "TotalUsageInHour"
    ]
}
```

Body Cookies Headers (7) Test Results 200 OK 783 ms 1.64 KB Save Response

JSON Visualize

UserPrincipalName	contactEmail	ServicePlanId	ServicePlanName	provisioningPolicyGroupId	provisioningPolicyGroupName	ExpectedEnterpriseLicenseGroupName
test0@cognitionittst.onmicros	test0@cognitionit.com	2de9c682-1234	Cloud PC Enterprise 4vCPU/16GB/128GE	736b9fc8-1234-1234-1234	W365-TST Test Cloud PC	W365-LIC-4vCPU-16GB-128GB
test1@cognitionittst.onmicros	test1@cognitionit.com	69dc175c-1234	Cloud PC Enterprise 8vCPU/32GB/256GE	736b9fc8-1234-1234-1234	W365-TST Test Cloud PC	W365-LIC-8vCPU-32GB-256GB
test0@cognitionitdev.onmicrc	test0@cognitionit.com	2de9c682-1234	Cloud PC Enterprise 4vCPU/16GB/128GE	b8483202-1234-1234-1234	W365-DEV Development Cloud PC	W365-LIC-4vCPU-16GB-128GB
test1@cognitionitdev.onmicrc	test1@cognitionit.com	057efbfe-1234-	Cloud PC Frontline 2vCPU/8GB/128GB	a0249b00-1234-1234-1234	W365-DEV Development Cloud PC FL	
test2@cognitionitdev.onmicrc	test2@cognitionit.com	3efff3fe-1234-1	Cloud PC Enterprise 2vCPU/8GB/128GB	b8483202-1234-1234-1234	W365-DEV Development Cloud PC	W365-LIC-2vCPU-8GB-128GB
test3@cognitionitdev.onmicrc	test3@cognitionit.com	057efbfe-1234-	Cloud PC Frontline 2vCPU/8GB/128GB	a0249b00-1234-1234-1234	W365-DEV Development Cloud PC FL	
test1@cognitionit.com	test1@cognitionit.com	057efbfe-1234-	Cloud PC Frontline 2vCPU/8GB/128GB	db320d66-1234-1234-1234	W365-PRD Production Cloud PC	
test2@cognitionit.com	test2@cognitionit.com	057efbfe-1234-	Cloud PC Frontline 2vCPU/8GB/128GB	db320d66-1234-1234-1234	W365-PRD Production Cloud PC	

W365 CloudPC License Management (filtered)



00. GraphAPI Demos / Windo... / 03. License Management / **CloudPCReports: getTotalAggregatedRemoteConnectionReports**

Save | Share

POST https://graph.microsoft.com/beta/deviceManagement/virtualEndpoint/reports/getTotalAggregatedRemoteConnectionReports/?$count=1... Send

Params • Authorization Headers (12) Body • Scripts Settings Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL JSON Beautify

```
{
    "skip": 0,
    "select": [],
    "top": 1,
    "search": "",
    "orderBy": [
        "TotalUsageInHour"
    ]
}
```

Body Cookies Headers (7) Test Results 200 OK • 783 ms • 1.64 KB Save Response

NeverSignedIn	LastActiveTime	CreatedDate	DaysSinceLastSignIn	UserPrincipalName	contactEmail	ServicePlanId	ServicePlanName
TRUE		2025-12-05 12:07		test0@cognitionittst.onmicro	test0@cognitionit.com	2de9c682-123	Cloud PC Enterprise 4vCPU/16GB/128GE
FALSE	6/20/2025 2:37:02 PM	2025-12-05 12:07	56	test1@cognitionittst.onmicro	test1@cognitionit.com	69dc175c-123	Cloud PC Enterprise 8vCPU/32GB/256GE
FALSE	7/16/2025 9:24:11 AM	7/14/2023 9:37:07 AM	30	test0@cognitionitdev.onmicrc	test0@cognitionit.com	2de9c682-123	Cloud PC Enterprise 4vCPU/16GB/128GE
FALSE	7/16/2025 5:55:31 PM	2025-09-07 08:38	30	test1@cognitionitdev.onmicrc	test1@cognitionit.com	057efbfe-1234-	Cloud PC Frontline 2vCPU/8GB/128GB
FALSE	6/30/2025 7:09:21 AM	2024-08-04 06:28	46	test2@cognitionitdev.onmicrc	test2@cognitionit.com	3efff3fe-1234-1	Cloud PC Enterprise 2vCPU/8GB/128GB
TRUE		8/13/2025 8:08:52 AM		test3@cognitionitdev.onmicrc	test3@cognitionit.com	057efbfe-1234-	Cloud PC Frontline 2vCPU/8GB/128GB
TRUE		2025-06-05 07:45		test1@cognitionit.com	test1@cognitionit.com	057efbfe-1234-	Cloud PC Frontline 2vCPU/8GB/128GB
FALSE	2025-11-07 12:57	11/14/2024 10:04:12 AM	35	test2@cognitionit.com	test2@cognitionit.com	057efbfe-1234-	Cloud PC Frontline 2vCPU/8GB/128GB

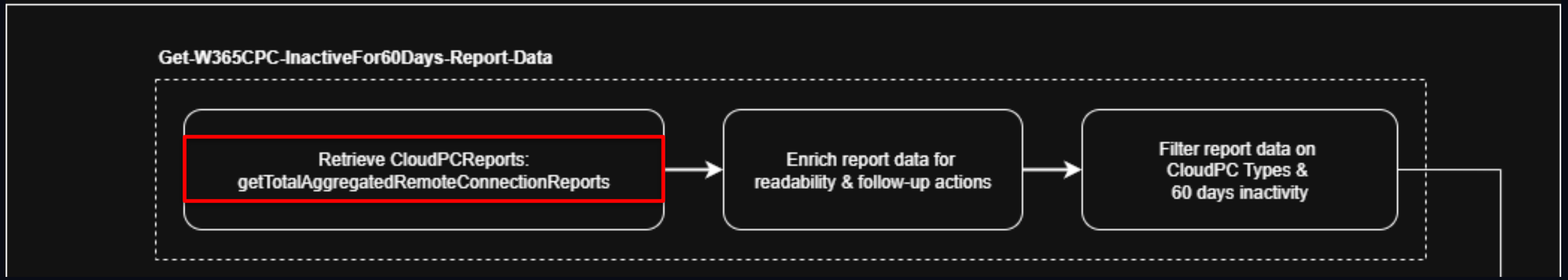
WORKPLACE NINJA SUMMIT

W365 License Management

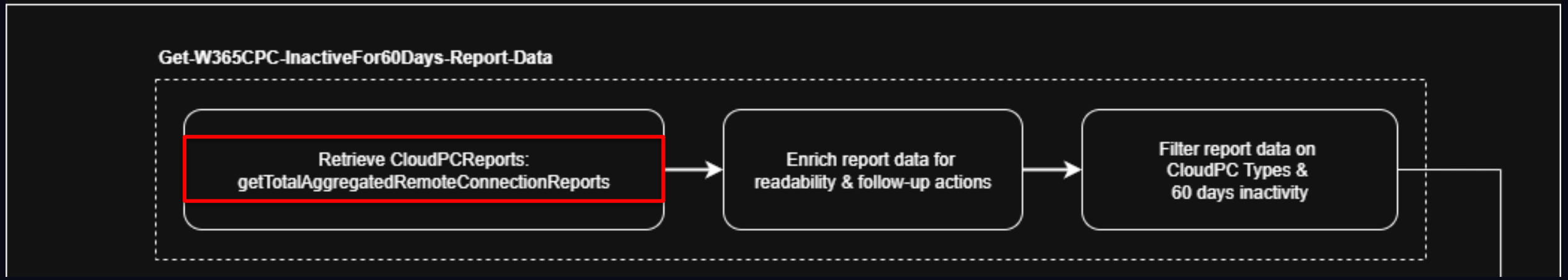
PowerShell script challenges



W365 CloudPC License Management



W365 CloudPC License Management



Challenges:

- response data pagination
- response formatting (PSObject)

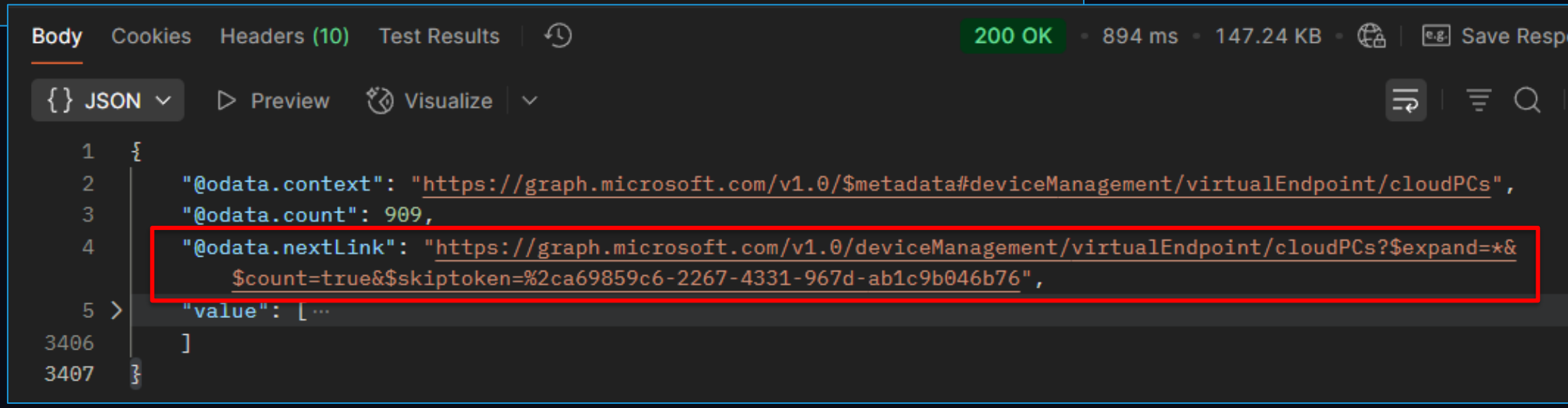
Graph API PowerShell challenge (GET pagination)

Paging Microsoft Graph data in your app

04/03/2025

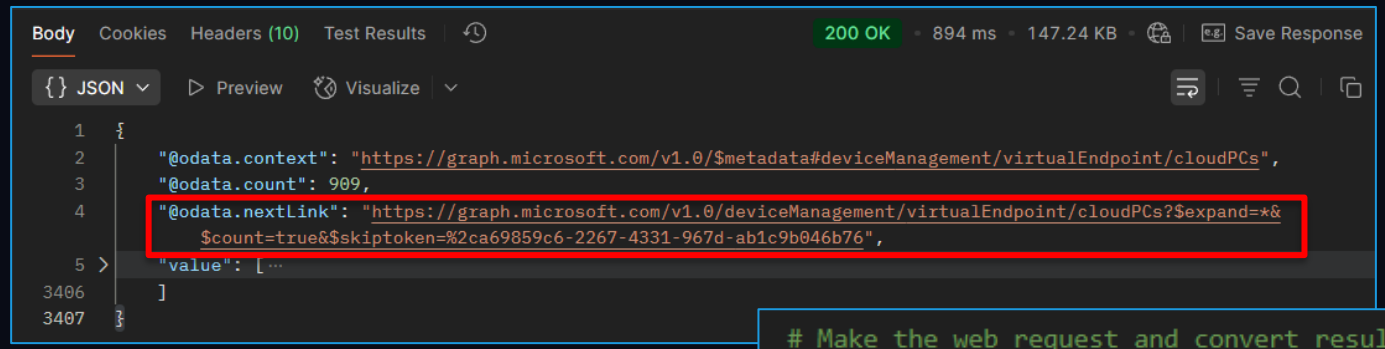
Paging involves requesting or receiving data in batches. It's a performance technique that's crucial for efficiently handling large datasets and that helps improve the performance of your app and the response time of Microsoft Graph.

Some GET queries against Microsoft Graph return multiple pages of data either due to server-side paging or client-side paging. In this article, we explore how paging works for Microsoft Graph and how you can use it to optimize your applications.



Paging Microsoft Graph data in your app - Microsoft Graph | Microsoft Learn

Graph API PowerShell challenge (GET pagination)



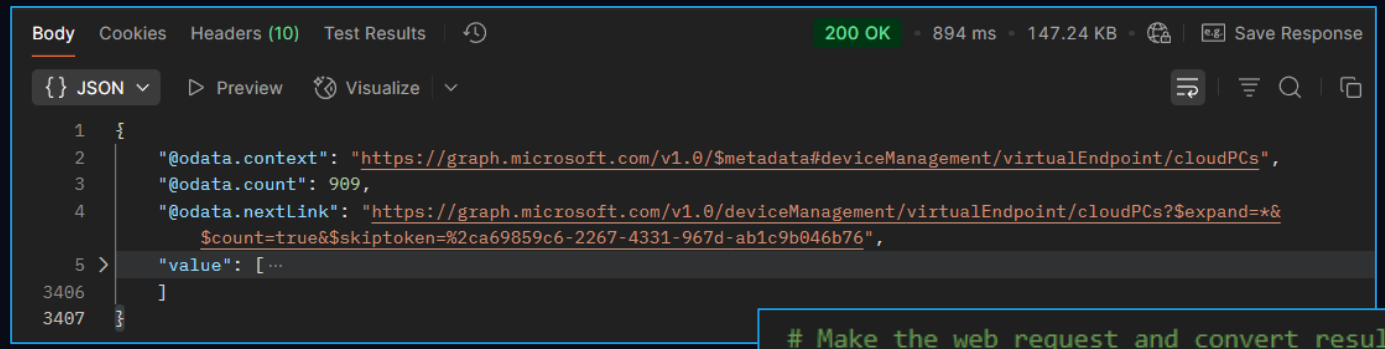

```powershell
# Make the web request and convert result content to PSObject to retrieve the Windows365 CloudPCs
$webrequestResult = ConvertFrom-Json -InputObject $($(Invoke-WebRequest @webRequestParams).Content)

# Create an array to store all CloudPCs
$allCloudPCs = @()
$allCloudPCs += $webrequestResult.value

# Resolve Graph API pagination to retrieve all CloudPCs
while ($null -ne $webrequestResult.'@odata.nextLink') {
    $uri = $webrequestResult.'@odata.nextLink'
    $webRequestParams = @{
        Uri     = $uri
        Headers = $authHeader
        Method  = $method
    }
    $webrequestResult = ConvertFrom-Json -InputObject $($(Invoke-WebRequest @webRequestParams).Content)
    # Store next page of CloudPCs in the allCloudPCs array
    $allCloudPCs += $($webrequestResult.value)
}
# Return the array of all CloudPCs
return $allCloudPCs
```

Paging Microsoft Graph data in your app - Microsoft Graph | Microsoft Learn

Graph API PowerShell challenge (GET pagination)




```powershell
# Make the web request and convert result content to PSObject to retrieve the Windows365 CloudPCs
$webrequestResult = ConvertFrom-Json -InputObject $($(Invoke-WebRequest @webRequestParams).Content)

# Create an array to store all CloudPCs
$allCloudPCs = @()
$allCloudPCs += $webrequestResult.value

# Resolve Graph API pagination to retrieve all CloudPCs
while ($null -ne $webrequestResult.'@odata.nextLink') {
    $uri = $webrequestResult.'@odata.nextLink'
    $webRequestParams = @{
        Uri     = $uri
        Headers = $authHeader
        Method  = $method
    }
    $webrequestResult = ConvertFrom-Json -InputObject $($(Invoke-WebRequest @webRequestParams).Content)
    # Store next page of CloudPCs in the allCloudPCs array
    $allCloudPCs += $($webrequestResult.value)
}
# Return the array of all CloudPCs
return $allCloudPCs
```

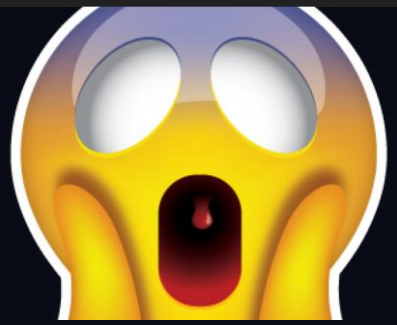
Paging Microsoft Graph data in your app - Microsoft Graph | Microsoft Learn

Graph API PowerShell challenge (POST pagination)

Body Cookies Headers (7) Test Results

{} JSON Visualize


```
1    {
2        "TotalRowCount": 906,
3    >   "Schema": [···
68       ],
69   >   "Values": [···
1870     ]
1871 }
```

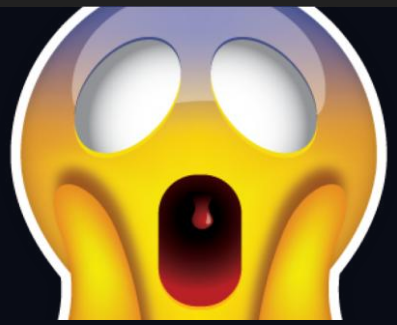
NO @odata.nextLink !!!!!

Graph API PowerShell challenge (POST pagination)

Body | Cookies | Headers (7) | Test Results

{} JSON | Visualize

```
1    {
2        "TotalRowCount": 906,
3    >   "Schema": [···
68       ],
69   >   "Values": [···
1870     ]
1871 }
```



NO @odata.nextLink !!!!!

Copilot

Paging data from the Microsoft Graph API using the `POST` method—especially for endpoints like **Cloud PC reports**—requires a slightly different approach than traditional `GET` requests. Here's how you can handle pagination using the `skip` parameter in this context:

📊 Scenario: Paging Cloud PC Reports via POST

Microsoft Graph's reporting endpoints (like `/reports/cloudPCPerformance`, `/reports/cloudPCConnection`, etc.) often use `POST` for complex queries. These endpoints typically support pagination via the `skip` parameter in the request body.

3. Loop Through Pages

To get all data, loop through the pages by increasing `skip`:

Python ^ Copy

```python
skip = 0
top = 100
results = []

while True:
    response = post_to_graph_api(skip=skip, top=top)
    data = response["value"]
    results.extend(data)

    if len(data) < top:
        break  # No more data
    skip += top
```

Graph API PowerShell challenge (POST pagination)

Body Cookies Headers (7) Test Results

{} JSON Visualize


```
{
    "TotalRowCount": 906,
    "Schema": [...
    ],
    "Values": [...
    ]
}
```




```powershell
# Create an array to store all AggregatedRemoteConnections
[array]$rawAggregatedRemoteConnections = @()

do {
    # Make the web request to retrieve the Total Aggregated Remote Connection Reports
    $webRequestParams = @{
        Uri     = $uri
        Headers = $authHeader
        Method  = $method
        Body    = $(ConvertTo-Json -InputObject $bodyObject -Depth 100)
    }

    # Use Invoke-RestMethod to handle base64-encoded webrequest.content
    $webrequestResults = Invoke-RestMethod @webRequestParams

    # Process the result of the web request
    [int]$totalColumns = ($webrequestResults.Schema | Measure-Object).Count
    foreach ($item in $webrequestResults.Values) {
        $aggregatedRemoteConnection = @{}
        for ($i = 0; $i -lt ($totalColumns - 1); $i++) {
            $aggregatedRemoteConnection.Add($webrequestResults.Schema[$i].Column, $item[$i])
        }
        $rawAggregatedRemoteConnections += $aggregatedRemoteConnection
    }

    # Update skip and check for pagination
    $retrievedRows = ($webrequestResults.Values | Measure-Object).Count
    $skip += $retrievedRows
    $totalRowCount = $webrequestResults.TotalRowCount

    # Update the request body for the next page
    $bodyObject["skip"] = $skip
} while ($skip -lt $totalRowCount)

# Return the array of all Aggregated Remote Connections
return $rawAggregatedRemoteConnections
```

Graph API PowerShell challenge (POST pagination)

Body | Cookies | Headers (7) | Test Results

{} JSON | Visualize

```
{
    "TotalRowCount": 906,
    "Schema": [...
    ],
    "Values": [...
    ]
}
```



```powershell
# Create an array to store all AggregatedRemoteConnections
[array]$rawAggregatedRemoteConnections = @()

do {
    # Make the web request to retrieve the Total Aggregated Remote Connection Reports
    $webRequestParams = @{
        Uri     = $uri
        Headers = $authHeader
        Method  = $method
        Body    = $(ConvertTo-Json -InputObject $bodyObject -Depth 100)
    }

    # Use Invoke-RestMethod to handle base64-encoded webrequest.content
    $webrequestResults = Invoke-RestMethod @webRequestParams

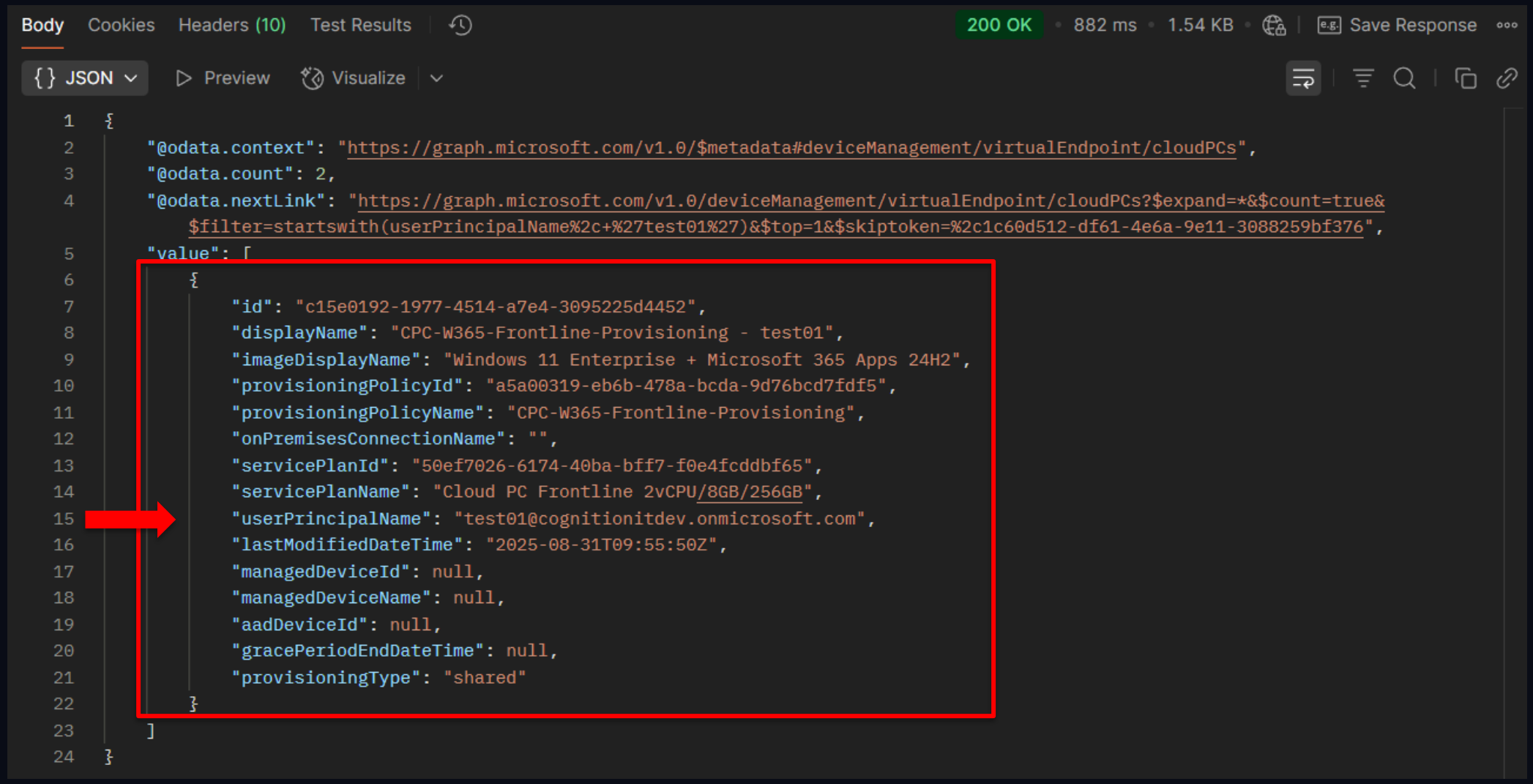
    # Process the result of the web request
    [int]$totalColumns = ($webrequestResults.Schema | Measure-Object).Count
    foreach ($item in $webrequestResults.Values) {
        $aggregatedRemoteConnection = @{}
        for ($i = 0; $i -lt ($totalColumns - 1); $i++) {
            $aggregatedRemoteConnection.Add($webrequestResults.Schema[$i].Column, $item[$i])
        }
        $rawAggregatedRemoteConnections += $aggregatedRemoteConnection
    }

    # Update skip and check for pagination
    $retrievedRows = ($webrequestResults.Values | Measure-Object).Count
    $skip += $retrievedRows
    $totalRowCount = $webrequestResults.TotalRowCount

    # Update the request body for the next page
    $bodyObject["skip"] = $skip
} while ($skip -lt $totalRowCount)

# Return the array of all Aggregated Remote Connections
return $rawAggregatedRemoteConnections
```

Graph API PowerShell challenge (response -> PSObject)



List cloudPCs - Microsoft Graph beta | Microsoft Learn

Graph API PowerShell challenge (response -> PSObject)

```powershell
# Make the web request and convert result content to PSObject to retrieve the Windows365 CloudPCs
$webrequestResult = ConvertFrom-Json -InputObject $($(Invoke-WebRequest @webRequestParams).Content)

# Create an array to store all CloudPCs
$allCloudPCs = @()
$allCloudPCs += $webrequestResult.value

# Resolve Graph API pagination to retrieve all CloudPCs
while ($null -ne $webrequestResult.'@odata.nextLink') {
    $uri = $webrequestResult.'@odata.nextLink'
    $webRequestParams = @{ ···
    }
    $webrequestResult = ConvertFrom-Json -InputObject $($(Invoke-WebRequest @webRequestParams).Content)
    # Store next page of CloudPCs in the allCloudPCs array
    $allCloudPCs += $($webrequestResult.value)
}
# Return the array of all CloudPCs
return $allCloudPCs
```

List cloudPCs - Microsoft Graph beta | Microsoft Learn

Graph API PowerShell challenge (response -> PSObject)

W365iicenseReport.ps1

C: > Temp > W365iicenseReport.ps1 > ...

```
591   $w365CloudPCs = Get-W365CloudPCs -AccessToken $($graphContext.access_token)
592
```

PROBLEMS TERMINAL PORTS GITLENS AZURE OUTPUT DEBUG CONSOLE

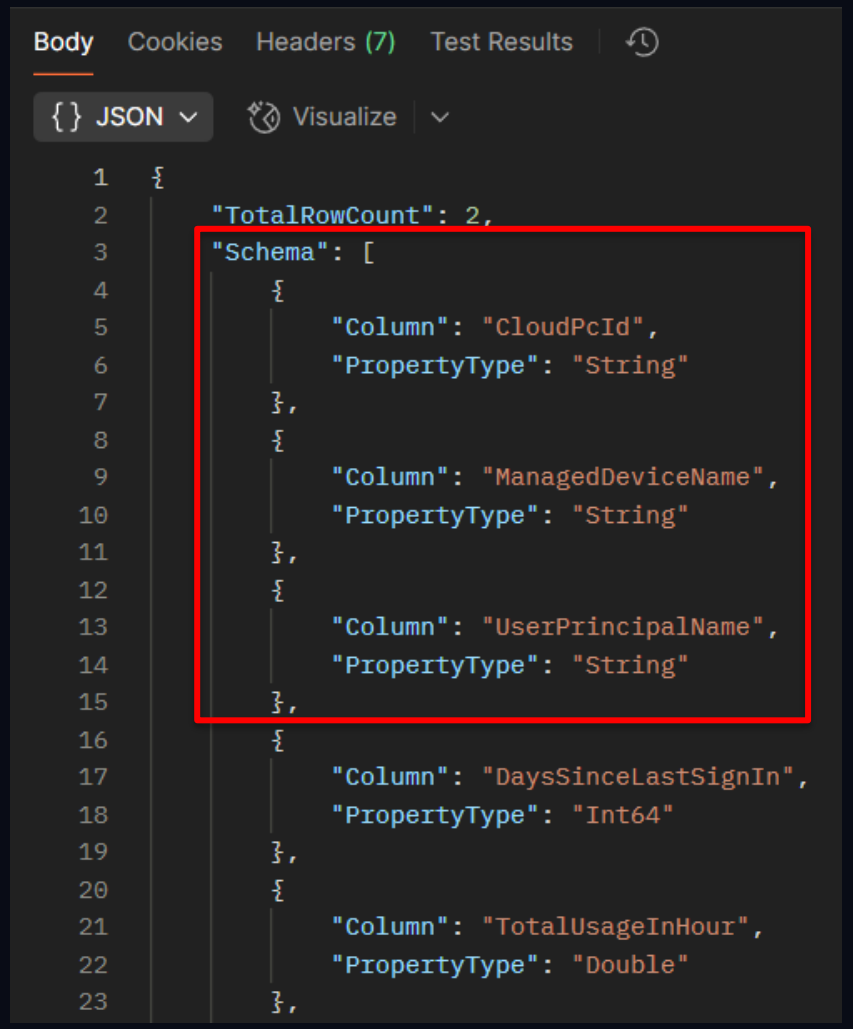
```
PS C:\Temp> $w365CloudPCs[0]

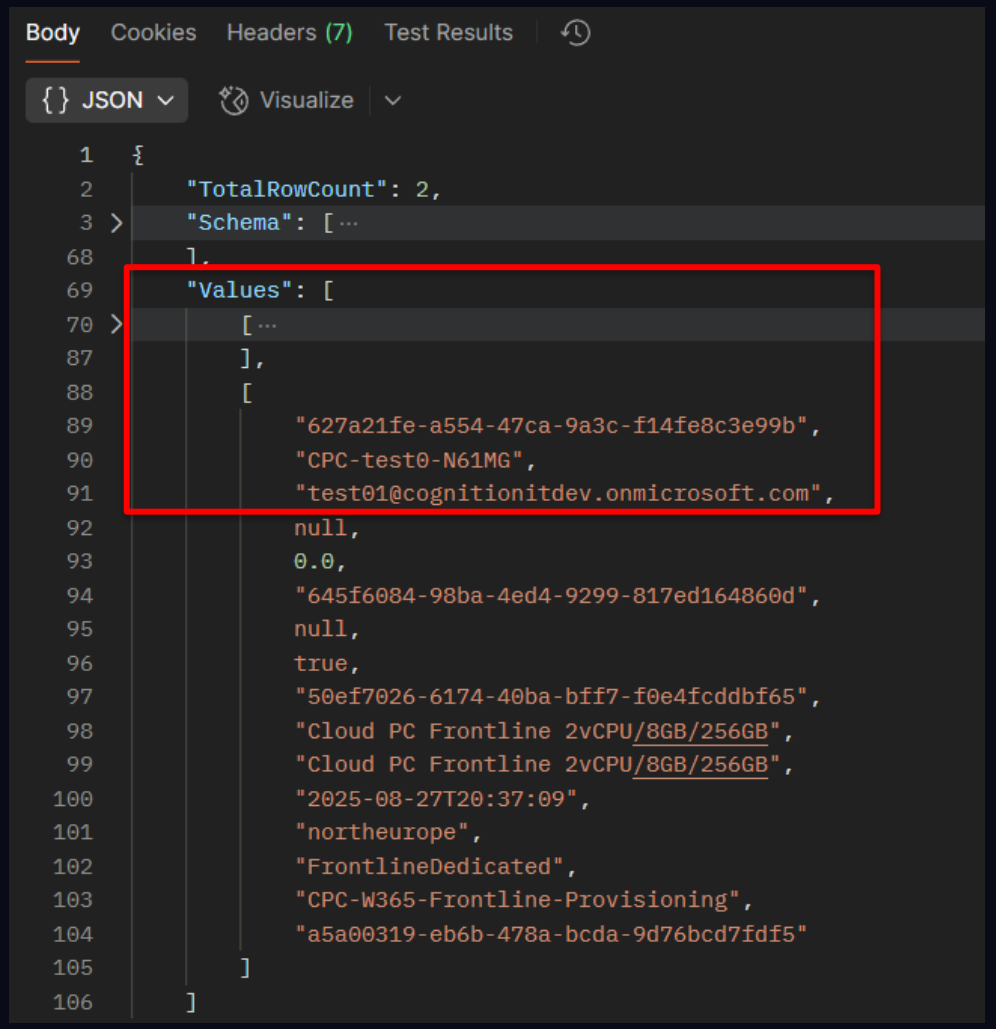
id                        : c15e0192-1977-4514-a7e4-3095225d4452
displayName               : CPC-W365-Frontline-Provisioning - test01
imageDisplayName          : Windows 11 Enterprise + Microsoft 365 Apps 24H2
provisioningPolicyId      : a5a00319-eb6b-478a-bcda-9d76bcd7fdf5
provisioningPolicyName    : CPC-W365-Frontline-Provisioning
onPremisesConnectionName  :
servicePlanId             : 50ef7026-6174-40ba-bff7-f0e4fcddbf65
servicePlanName           : Cloud PC Frontline 2vCPU/8GB/256GB
status                    : provisioning
userPrincipalName         : test01@cognitionitdev.onmicrosoft.com
lastModifiedDateTime      : 2025-08-31 09:53:38
managedDeviceId           :
managedDeviceName         :
aadDeviceId               :
gracePeriodEndDateTime    :
servicePlanType           : enterprise
diskEncryptionState       : notAvailable
provisioningType          : sharedByUser
statusDetails             :
statusDetail              :
partnerAgentInstallResults : {}

PS C:\Temp>
```

List cloudPCs - Microsoft Graph beta | Microsoft Learn

Graph API PowerShell challenge (response -> PSObject)





cloudPcReports: getTotalAggregatedRemoteConnectionReports - Microsoft Graph beta | Microsoft Learn

Graph API PowerShell challenge (response -> PSObject)

```powershell
# Create an array to store all AggregatedRemoteConnections
[array]$rawAggregatedRemoteConnections = @()

do {
    # Make the web request to retrieve the Total Aggregated Remote Connection Reports
    $webRequestParams = @{ ···
    }

    # Use Invoke-RestMethod to handle base64-encoded webrequest.content
    $webrequestResults = Invoke-RestMethod @webRequestParams

    # Process the result of the web request
    [int]$totalColumns = ($webrequestResults.Schema | Measure-Object).Count
    foreach ($item in $webrequestResults.Values) {
        $aggregatedRemoteConnection = @{}
        for ($i = 0; $i -lt ($totalColumns - 1); $i++) {
            $aggregatedRemoteConnection.Add($webrequestResults.Schema[$i].Column, $item[$i])
        }
        $rawAggregatedRemoteConnections += $aggregatedRemoteConnection
    }
} while ($skip -lt $totalRowCount)

# Return the array of all Aggregated Remote Connections
return $rawAggregatedRemoteConnections
```

cloudPcReports: getTotalAggregatedRemoteConnectionReports - Microsoft Graph beta | Microsoft Learn

Graph API PowerShell challenge (response -> PSObject)

```powershell
# Create an array to store all AggregatedRemoteConnections
[array]$rawAggregatedRemoteConnections = @()

do {
    # Make the web request to retrieve the Total Aggregated Remote Connection Reports
    $webRequestParams = @{ ···
    }

    # Use Invoke-RestMethod to handle base64-encoded webrequest.content
    $webrequestResults = Invoke-RestMethod @webRequestParams

    # Process the result of the web request
    [int]$totalColumns = ($webrequestResults.Schema | Measure-Object).Count
    foreach ($item in $webrequestResults.Values) {
        $aggregatedRemoteConnection = @{}
        for ($i = 0; $i -lt ($totalColumns - 1); $i++) {
            $aggregatedRemoteConnection.Add($webrequestResults.Schema[$i].Column, $item[$i])
        }
        $rawAggregatedRemoteConnections += $aggregatedRemoteConnection
    }
} while ($skip -lt $totalRowCount)

# Return the array of all Aggregated Remote Connections
return $rawAggregatedRemoteConnections
```

```
"Schema": [
    {
        "Column": "CloudPcId",
        "PropertyType": "String"
    },
    {
[1]     "Column": "ManagedDeviceName",
        "PropertyType": "String"
    },
```

```
"Values": [
    [···
    ],
    [
        "627a21fe-a554-47ca-9a3c-f14fe8c3e99b",
[1]     "CPC-test0-N61MG",
```

cloudPcReports: getTotalAggregatedRemoteConnectionReports - Microsoft Graph beta | Microsoft Learn

Graph API PowerShell challenge (response -> PSObject)

C: > Temp > W365iicenseReport.ps1 > ...

```
607   # Get Intune report data for Total Aggregated Remote Connections
608   $rawAggregatedRemoteConnections = Get-W365AggregatedRemoteConnectionReport -AccessToken $($graphContext.access_token)
609   $enrichedAggregatedRemoteConnections = Get-W365AggregatedRemoteConnectionReportWithEnrichedData -AccessToken $($graphContext.access_token) `
610                                          -ServicePlanSearchtable $w365ServicePlanSearchtable -LicenseGroupSearchtable $w365LicenseGroupsSearchTable `
611                                          -CloudPcSearchtable $w365CloudPCs -ProvisioningPolicySearchtable $w365ProvisioningPoliciesSearchTable
612
```

PROBLEMS TERMINAL PORTS GITLENS AZURE OUTPUT DEBUG CONSOLE

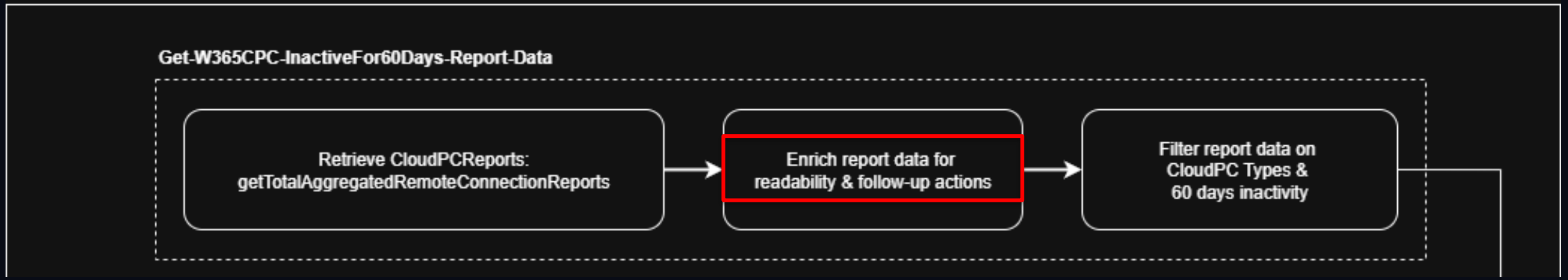
```
PS C:\Temp> $rawAggregatedRemoteConnections[1] | ConvertTo-Json -Depth 10 | ConvertFrom-Json

LastActiveTime      :
CreatedDate         : 2025-08-27 20:37:09
ManagedDeviceName   : CPC-test0-N61MG
ServicePlanId       : 50ef7026-6174-40ba-bff7-f0e4fcddbf65
UserPrincipalName   : test01@cognitionitdev.onmicrosoft.com
CloudPcId           : 627a21fe-a554-47ca-9a3c-f14fe8c3e99b
DeviceType          : FrontlineDedicated
CloudPCSize         : Cloud PC Frontline 2vCPU/8GB/256GB
IntuneDeviceId      : 645f6084-98ba-4ed4-9299-817ed164860d
PcType              : Cloud PC Frontline 2vCPU/8GB/256GB
DaysSinceLastSignIn :
PolicyName          : CPC-W365-Frontline-Provisioning
NeverSignedIn       : True
HostRegion          : northeurope
TotalUsageInHour    : 0

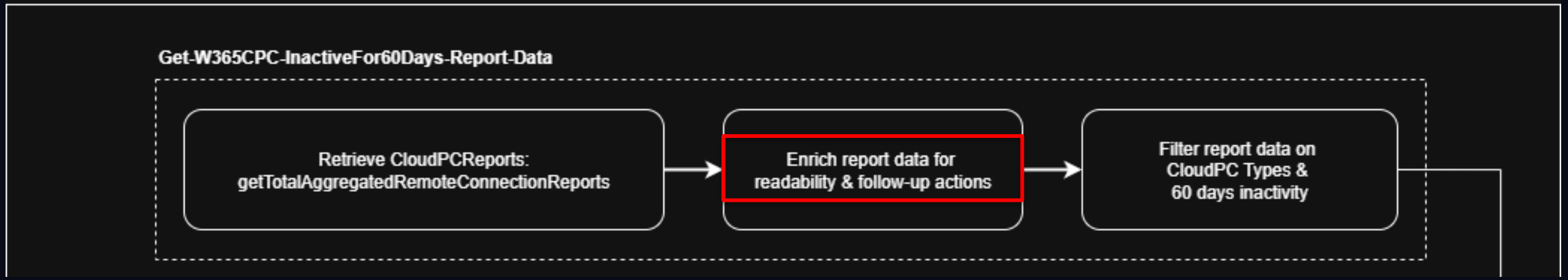
PS C:\Temp>
```

cloudPcReports: getTotalAggregatedRemoteConnectionReports - Microsoft Graph beta | Microsoft Learn

W365 CloudPC License Management



W365 CloudPC License Management



Challenges:

- add data to PSObject
- optimize additional API calls

Graph API PowerShell challenge (enrich PSObject)

```powershell
# Create an array to store all AggregatedRemoteConnections
[array]$rawAggregatedRemoteConnections = @()

do {
    # Make the web request to retrieve the Total Aggregated Remote Connection Reports
    $webRequestParams = @{ ···
    }

    # Use Invoke-RestMethod to handle base64-encoded webrequest.content
    $webrequestResults = Invoke-RestMethod @webRequestParams

    # Process the result of the web request
    [int]$totalColumns = ($webrequestResults.Schema | Measure-Object).Count
    foreach ($item in $webrequestResults.Values) {
        $aggregatedRemoteConnection = @{}
        for ($i = 0; $i -lt ($totalColumns - 1); $i++) {
            $aggregatedRemoteConnection.Add($webrequestResults.Schema[$i].Column, $item[$i])
        }
        $rawAggregatedRemoteConnections += $aggregatedRemoteConnection
    }
} while ($skip -lt $totalRowCount)

# Return the array of all Aggregated Remote Connections
return $rawAggregatedRemoteConnections
```

cloudPcReports: getTotalAggregatedRemoteConnectionReports - Microsoft Graph beta | Microsoft Learn

Graph API PowerShell challenge (enrich PSObject)

```powershell
# Process the result of the web request
[int]$totalColumns = ($webrequestResults.Schema | Measure-Object).Count
foreach ($item in $webrequestResults.Values) {
    $aggregatedRemoteConnection = @{}
    for ($i = 0; $i -lt ($totalColumns - 1); $i++) {
        $aggregatedRemoteConnection.Add($webrequestResults.Schema[$i].Column, $item[$i])

        # Add ServicePlan details
        if ($webrequestResults.Schema[$i].Column -eq 'ServicePlanId') {
            $servicePlan = $ServicePlanSearchtable.where({ $_.id -eq $item[$i] })
            $aggregatedRemoteConnection.Add('ServicePlanName', $servicePlan.displayName)
            $aggregatedRemoteConnection.Add('ServicePlanType', $servicePlan.type)
            # Add ServicePlanProvisioningType and ExpectedEnterpriseLicenseGroupName
            switch ($servicePlan.provisioningType) {
                'shared' {
                    $ServicePlanProvisioningType = 'Frontline'
                    $EnterpriseLicenseGroupName = ''
                }
                'dedicated' {
                    $ServicePlanProvisioningType = 'Enterprise'
                    $EnterpriseLicenseGroupName = "W365-LIC-$($servicePlan.vCpuCount)vCPU-"
                    $EnterpriseLicenseGroupName += "$($servicePlan.ramInGB)GB-"
                    $EnterpriseLicenseGroupName += "$($servicePlan.storageInGB)GB"

                }
                default {
                    $ServicePlanProvisioningType = 'Unknown'
                    $EnterpriseLicenseGroupName = ''
                }
            }
            $aggregatedRemoteConnection.Add('ServicePlanProvisioningType', $ServicePlanProvisioningType)
            $aggregatedRemoteConnection.Add('ExpectedEnterpriseLicenseGroupName', $EnterpriseLicenseGroupName)
            # search EnterpriseLicenseGroupId in $w365LicenseGroupsSearchTable
            $licenseGroup = $LicenseGroupSearchtable.where({ $_.displayName -eq $EnterpriseLicenseGroupName })
            $aggregatedRemoteConnection.Add('ExpectedEnterpriseLicenseGroupId', $($licenseGroup.id))
        }

        # Add CloudPC details
        if ($webrequestResults.Schema[$i].Column -eq 'CloudPcId') {
            $cloudPC = $CloudPcSearchtable.where({ $_.id -eq $item[$i] })
            $aggregatedRemoteConnection.Add('CloudPCName', $cloudPC.displayName)
            $aggregatedRemoteConnection.Add('ProvisioningPolicyId', $cloudPC.provisioningPolicyId)
            $aggregatedRemoteConnection.Add('ProvisioningPolicyName', $cloudPC.provisioningPolicyName)
            $aggregatedRemoteConnection.Add('provisioningType', $cloudPC.provisioningType)
            $aggregatedRemoteConnection.Add('lastModifiedDateTime', $cloudPC.lastModifiedDateTime)
            $aggregatedRemoteConnection.Add('provisioningStatus', $cloudPC.status)
            # Add provisioning policy groupname & groupid to dataset
            $assignmentServicePlanId = $cloudPC.servicePlanId
            $asignmentProvisioningPolicyId = $cloudPC.provisioningPolicyId
            $provisioningPolicy = $ProvisioningPolicySearchtable.where({ ($_.id -eq $asignmentProvisioningPolicyId) `
                        -and ( ($_.assignmentServicePlanId -eq $assignmentServicePlanId) -or ($null -eq $_.assignmentServicePlanId) ) })
            $aggregatedRemoteConnection.Add('provisioningPolicyGroupId', $provisioningPolicy.assignmentGroupId)
            $aggregatedRemoteConnection.Add('provisioningPolicyGroupName', $provisioningPolicy.assignmentGroupName)
            $aggregatedRemoteConnection.Add('aadDeviceId', $cloudPC.aadDeviceId)
            # Convert UPN to contact email address
            $userPrincipalName = $cloudPC.userPrincipalName
            if ($userPrincipalName.endsWith('cognitionitdev.onmicrosoft.com')) {
                $userPrincipalName = $userPrincipalName.Replace('cognitionitdev.onmicrosoft.com', 'cognitionit.com')
            }
            if ($userPrincipalName.endsWith('cognitionittst.onmicrosoft.com')) {
                $userPrincipalName = $userPrincipalName.Replace('cognitionittst.onmicrosoft.com', 'cognitionit.com')
            }
            $contactEmail = $userPrincipalName
            $aggregatedRemoteConnection.Add('contactEmail', $contactEmail)
        }
    }
    $allAggregatedRemoteConnections += $aggregatedRemoteConnection
}
```

cloudPcReports: getTotalAggregatedRemoteConnectionReports - Microsoft Graph beta | Microsoft Learn

Graph API PowerShell challenge (enrich PSObject)

```powershell
# Process the result of the web request
[int]$totalColumns = ($webrequestResults.Schema | Measure-Object).Count
foreach ($item in $webrequestResults.Values) {
    $aggregatedRemoteConnection = @{}
    for ($i = 0; $i -lt ($totalColumns - 1); $i++) {
        $aggregatedRemoteConnection.Add($webrequestResults.Schema[$i].Column, $item[$i])

        # Add ServicePlan details
        if ($webrequestResults.Schema[$i].Column -eq 'ServicePlanId') {
            $servicePlan = $ServicePlanSearchtable.where({ $_.id -eq $item[$i] })
            $aggregatedRemoteConnection.Add('ServicePlanName', $servicePlan.displayName)
            $aggregatedRemoteConnection.Add('ServicePlanType', $servicePlan.type)
```

cloudPcReports: getTotalAggregatedRemoteConnectionReports - Microsoft Graph beta | Microsoft Learn

Graph API PowerShell challenge (enrich PSObject)

```powershell
# Process the result of the web request
[int]$totalColumns = ($webrequestResults.Schema | Measure-Object).Count
foreach ($item in $webrequestResults.Values) {
    $aggregatedRemoteConnection = @{}
    for ($i = 0; $i -lt ($totalColumns - 1); $i++) {
        $aggregatedRemoteConnection.Add($webrequestResults.Schema[$i].Column, $item[$i])

        # Add ServicePlan details
        if ($webrequestResults.Schema[$i].Column -eq 'ServicePlanId') {
            $servicePlan = $ServicePlanSearchtable.where({ $_.id -eq $item[$i] })
            $aggregatedRemoteConnection.Add('ServicePlanName', $servicePlan.displayName)
            $aggregatedRemoteConnection.Add('ServicePlanType', $servicePlan.type)
```

cloudPcReports: getTotalAggregatedRemoteConnectionReports - Microsoft Graph beta | Microsoft Learn

Graph API PowerShell challenge (enrich PSObject)

```powershell
# Process the result of the web request
[int]$totalColumns = ($webrequestResults.Schema | Measure-Object).Count
foreach ($item in $webrequestResults.Values) {
    $aggregatedRemoteConnection = @{}
    for ($i = 0; $i -lt ($totalColumns - 1); $i++) {
        $aggregatedRemoteConnection.Add($webrequestResults.Schema[$i].Column, $item[$i])

        # Add ServicePlan details
        if ($webrequestResults.Schema[$i].Column -eq 'ServicePlanId') {
            $servicePlan = $ServicePlanSearchtable.where({ $_.id -eq $item[$i] })
            $aggregatedRemoteConnection.Add('ServicePlanName', $servicePlan.displayName)
            $aggregatedRemoteConnection.Add('ServicePlanType', $servicePlan.type)
```

cloudPcReports: getTotalAggregatedRemoteConnectionReports - Microsoft Graph beta | Microsoft Learn

Graph API PowerShell challenge (enrich PSObject)

```powershell
# Process the result of the web request
[int]$totalColumns = ($webrequestResults.Schema | Measure-Object).Count
foreach ($item in $webrequestResults.Values) {
    $aggregatedRemoteConnection = @{}
    for ($i = 0; $i -lt ($totalColumns - 1); $i++) {
        $aggregatedRemoteConnection.Add($webrequestResults.Schema[$i].Column, $item[$i])

        # Add ServicePlan details
        if ($webrequestResults.Schema[$i].Column -eq 'ServicePlanId') {
            $servicePlan = $ServicePlanSearchtable.where({ $_.id -eq $item[$i] })
            $aggregatedRemoteConnection.Add('ServicePlanName', $servicePlan.displayName)
            $aggregatedRemoteConnection.Add('ServicePlanType', $servicePlan.type)
```

```
CloudPcId          : 627a21fe-a554-47ca-9a3c-f14fe8c3e99b
ManagedDeviceName  : CPC-test0-N61MG
UserPrincipalName  : test01@cognitionitdev.onmicrosoft.com
ServicePlanId      : 50ef7026-6174-40ba-bff7-f0e4fcddbf65
ServicePlanName    : Cloud PC Frontline 2vCPU/8GB/256GB
ServicePlanType    : enterprise
```

cloudPcReports: getTotalAggregatedRemoteConnectionReports - Microsoft Graph beta | Microsoft Learn

Graph API PowerShell challenge (enrich PSObject)

```powershell
        # Add ServicePlanProvisioningType and ExpectedEnterpriseLicenseGroupName
        switch ($servicePlan.provisioningType) {
            'shared' {
                $ServicePlanProvisioningType = 'Frontline'
                $EnterpriseLicenseGroupName = ''
            }
            'dedicated' {
                $ServicePlanProvisioningType = 'Enterprise'
                $EnterpriseLicenseGroupName = "W365-LIC-$($servicePlan.vCpuCount)vCPU-"
                $EnterpriseLicenseGroupName += "$($servicePlan.ramInGB)GB-"
                $EnterpriseLicenseGroupName += "$($servicePlan.storageInGB)GB"

            }
            default {
                $ServicePlanProvisioningType = 'Unknown'
                $EnterpriseLicenseGroupName = ''
            }
        }
        $aggregatedRemoteConnection.Add('ServicePlanProvisioningType', $ServicePlanProvisioningType)
        $aggregatedRemoteConnection.Add('ExpectedEnterpriseLicenseGroupName', $EnterpriseLicenseGroupName)
        # search EnterpriseLicenseGroupId in $w365LicenseGroupsSearchTable
        $licenseGroup = $LicenseGroupSearchtable.where({ $_.displayName -eq $EnterpriseLicenseGroupName })
        $aggregatedRemoteConnection.Add('ExpectedEnterpriseLicenseGroupId', $($licenseGroup.id))
    }
```

```
CloudPcId                       : 627a21fe-a554-47ca-9a3c-f14fe8c3e99b
ManagedDeviceName               : CPC-test0-N61MG
UserPrincipalName               : test01@cognitionitdev.onmicrosoft.com
ServicePlanProvisioningType     : Enterprise
ExpectedEnterpriseLicenseGroupName : W365-LIC-2vCPU-8GB-256GB
ExpectedEnterpriseLicenseGroupId   : 12345678-1234-1234-1234-123456789012
```

Graph API PowerShell challenge (enrich PSObject)

```powershell
# Add CloudPC details
if ($webrequestResults.Schema[$i].Column -eq 'CloudPcId') {
    $cloudPC = $CloudPcSearchtable.where({ $_.id -eq $item[$i] })
    $aggregatedRemoteConnection.Add('CloudPCName', $cloudPC.displayName)
    $aggregatedRemoteConnection.Add('ProvisioningPolicyId', $cloudPC.provisioningPolicyId)
    $aggregatedRemoteConnection.Add('ProvisioningPolicyName', $cloudPC.provisioningPolicyName)
    $aggregatedRemoteConnection.Add('provisioningType', $cloudPC.provisioningType)
    $aggregatedRemoteConnection.Add('lastModifiedDateTime', $cloudPC.lastModifiedDateTime)
    $aggregatedRemoteConnection.Add('provisioningStatus', $cloudPC.status)
    # Add provisioning policy groupname & groupid to dataset
    $assignmentServicePlanId = $cloudPC.servicePlanId
    $asignmentProvisioningPolicyId = $cloudPC.provisioningPolicyId
    $provisioningPolicy = $ProvisioningPolicySearchtable.where({ ($_.id -eq $asignmentProvisioningPolicyId) `
            -and ( ($_.assignmentServicePlanId -eq $assignmentServicePlanId) -or ($null -eq $_.assignmentServicePlanId) ) })
    $aggregatedRemoteConnection.Add('provisioningPolicyGroupId', $provisioningPolicy.assignmentGroupId)
    $aggregatedRemoteConnection.Add('provisioningPolicyGroupName', $provisioningPolicy.assignmentGroupName)
    $aggregatedRemoteConnection.Add('aadDeviceId', $cloudPC.aadDeviceId)
    # Convert UPN to contact email address
    $userPrincipalName = $cloudPC.userPrincipalName
    if ($userPrincipalName.endsWith('cognitionitdev.onmicrosoft.com')) {
        $userPrincipalName = $userPrincipalName.Replace('cognitionitdev.onmicrosoft.com', 'cognitionit.com')
    }
    if ($userPrincipalName.endsWith('cognitionittst.onmicrosoft.com')) {
        $userPrincipalName = $userPrincipalName.Replace('cognitionittst.onmicrosoft.com', 'cognitionit.com')
    }
    $contactEmail = $userPrincipalName
    $aggregatedRemoteConnection.Add('contactEmail', $contactEmail)
}
```



```
CloudPcId                 : 627a21fe-a554-47ca-9a3c-f14fe8c3e99b
CloudPCName               : CPC-W365-Frontline-Provisioning - test01
ProvisioningPolicyId      : a5a00319-eb6b-478a-bcda-9d76bcd7fdf5
ProvisioningPolicyName    : CPC-W365-Frontline-Provisioning
provisioningType          : sharedByUser
lastModifiedDateTime      : 2025-08-27 23:14:00
provisioningStatus        : provisioned
ProvisioningPolicyGroupId : 12345678-1234-1234-1234-123456789012
provisioningPolicyGroupName : {$null, grp-prv-W365-DEV-Users}
aadDeviceId               : 12345678-1234-1234-1234-123456789012
UserPrincipalName         : test01@cognitionitdev.onmicrosoft.com
contactEmail              : test01@cognitionit.com
```

Graph API PowerShell challenge (searchtables)

C: > Temp > W365iicenseReport.ps1 > function Get-W365AggregatedRemoteConnectionReportWithEnrichedData ()

```
394   function Get-W365AggregatedRemoteConnectionReportWithEnrichedData {
409       #region Create URI for Graph call to collect the Total Aggregated Remote Connection Reports
455
456               # Add ServicePlan details
457               if ($webrequestResults.Schema[$i].Column -eq 'ServicePlanId') {
458                   $servicePlan = $ServicePlanSearchtable.where({ $_.id -eq $item[$i] })
459                   $aggregatedRemoteConnection.Add('ServicePlanName', $servicePlan.displayName)
460                   $aggregatedRemoteConnection.Add('ServicePlanType', $servicePlan.type)
461
```

PROBLEMS TERMINAL PORTS GITLENS AZURE OUTPUT DEBUG CONSOLE

```
PS C:\Temp> $w365ServicePlanSearchtable.Where({$_.id -eq '50ef7026-6174-40ba-bff7-f0e4fcddbf65'})

id                : 50ef7026-6174-40ba-bff7-f0e4fcddbf65
displayName       : Cloud PC Frontline 2vCPU/8GB/256GB
type              : enterprise
vCpuCount         : 2
ramInGB           : 8
storageInGB       : 256
userProfileInGB   : 25
supportedSolution : windows365
provisioningType  : shared

PS C:\Temp>
```

List cloudPCs - Microsoft Graph beta | Microsoft Learn

Graph API PowerShell challenge (searchtables)

```powershell
# Build the Searchtables
$w365ServicePlanSearchtable = Get-W365ServicePlans -AccessToken $($graphContext.access_token)
$w365ProvisioningPolicyGroupSearchtable = Get-W365ProvisioningPolicyGroups -AccessToken $($graphContext.access_token) `
    -ProvisioningPolicyGroupDisplayName "grp-prv-W365"
$w365LicenseGroupsSearchtable = Get-W365LicenseGroups -AccessToken $($graphContext.access_token) `
    -LicenseGroupDisplayName "grp-lic-W365"
$w365ProvisioningPoliciesSearchtable = Get-W365ProvisioningPolicies -AccessToken $($graphContext.access_token) `
    -ProvisioningPolicyGroupSearchTable $w365ProvisioningPolicyGroupSearchtable `
    -ServicePlanSearchTable $w365ServicePlanSearchtable
$w365CloudPCs = Get-W365CloudPCs -AccessToken $($graphContext.access_token)
```

List cloudPCs - Microsoft Graph beta | Microsoft Learn

Graph API PowerShell challenge (searchtables)

```powershell
# Build the Searchtables
$w365ServicePlanSearchtable = Get-W365ServicePlans -AccessToken $($graphContext.access_token)
$w365ProvisioningPolicyGroupSearchtable = Get-W365ProvisioningPolicyGroups -AccessToken $($graphContext.access_token) `
    -ProvisioningPolicyGroupDisplayName "grp-prv-W365"
$w365LicenseGroupsSearchtable = Get-W365LicenseGroups -AccessToken $($graphContext.access_token) `
    -LicenseGroupDisplayName "grp-lic-W365"
$w365ProvisioningPoliciesSearchtable = Get-W365ProvisioningPolicies -AccessToken $($graphContext.access_token) `
    -ProvisioningPolicyGroupSearchTable $w365ProvisioningPolicyGroupSearchtable `
    -ServicePlanSearchTable $w365ServicePlanSearchtable
$w365CloudPCs = Get-W365CloudPCs -AccessToken $($graphContext.access_token)
```

PROBLEMS TERMINAL PORTS GITLENS AZURE OUTPUT DEBUG CONSOLE

```
Windows365 Service Plan SearchTable: 52
Windows365 Provisioning Policy Group SearchTable: 1
Windows365 License Groups SearchTable: 1
Windows365 Provisioning Policies SearchTable: 2
Windows365 CloudPCs SearchTable: 3
PS C:\Temp>
```

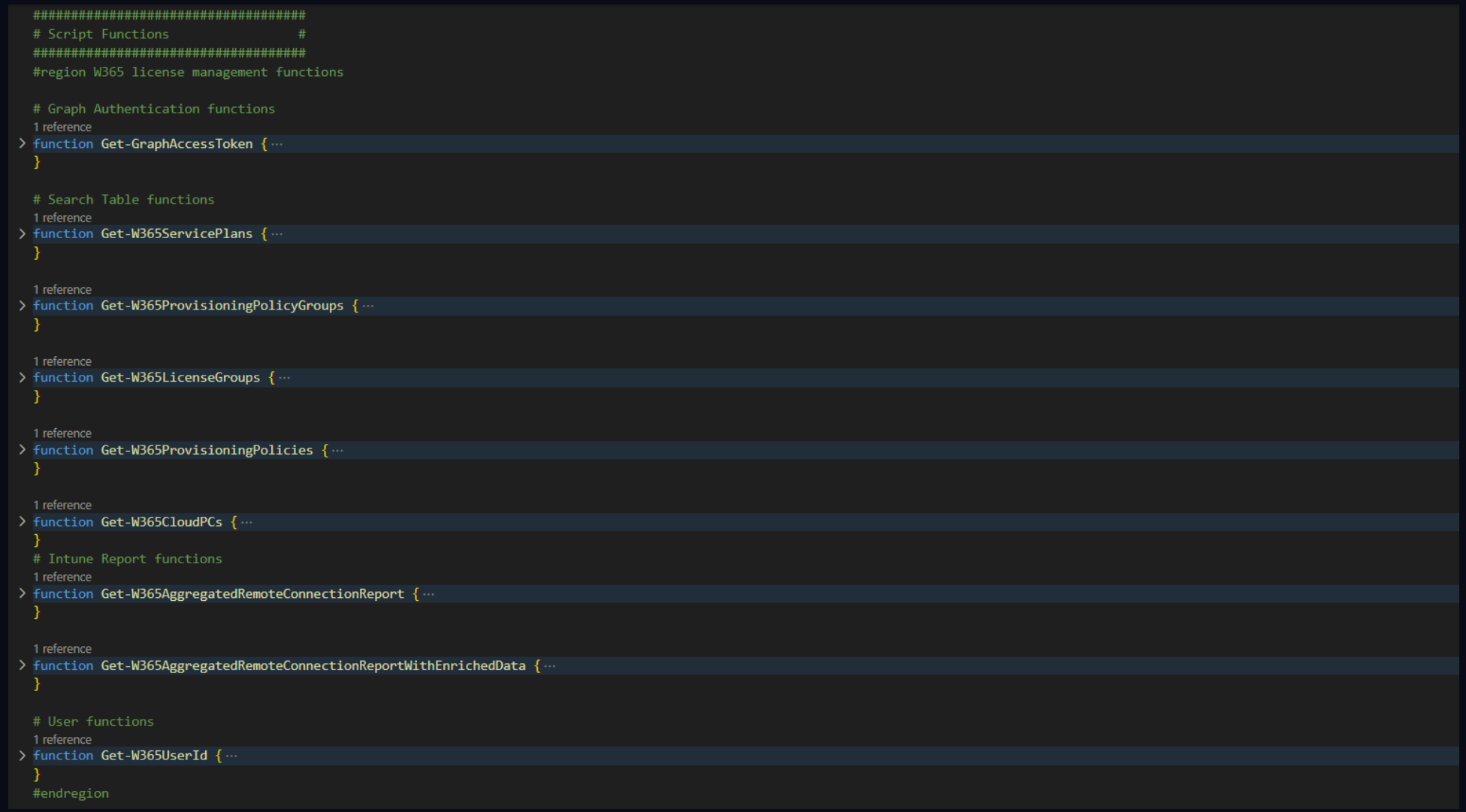
List cloudPCs - Microsoft Graph beta | Microsoft Learn

W365 License Management

PowerShell script



Graph API PowerShell script (functions)



Graph API PowerShell script (workflow)




```powershell
##################################
# Script workflow                #
##################################
# Get the access token
$graphContext = Get-GraphAccessToken -TenantId $tenantId -AppId $ClientId -AppSecret $ClientSecret
# Build the Searchtables
$w365ServicePlanSearchtable = Get-W365ServicePlans -AccessToken $($graphContext.access_token)
$w365ProvisioningPolicyGroupSearchtable = Get-W365ProvisioningPolicyGroups -AccessToken $($graphContext.access_token) -ProvisioningPolicyGroupDisplayName "grp-prv-W365"
$w365LicenseGroupsSearchtable = Get-W365LicenseGroups -AccessToken $($graphContext.access_token) -LicenseGroupDisplayName "grp-lic-W365"
$w365ProvisioningPoliciesSearchtable = Get-W365ProvisioningPolicies -AccessToken $($graphContext.access_token) -ProvisioningPolicyGroupSearchTable $w365ProvisioningPolicyGroupSearchtable -ServicePlanSearchTable $w365ServicePlanSearchtable
$w365CloudPCs = Get-W365CloudPCs -AccessToken $($graphContext.access_token)

#-------------------------------------------------------------------------#
# Retrieve Intune Total Aggregated Remote Connection Report for Windows365  #
#-------------------------------------------------------------------------#
# Get Intune report data for Total Aggregated Remote Connections
$rawAggregatedRemoteConnections = Get-W365AggregatedRemoteConnectionReport -AccessToken $($graphContext.access_token)
$enrichedAggregatedRemoteConnections = Get-W365AggregatedRemoteConnectionReportWithEnrichedData -AccessToken $($graphContext.access_token) -ServicePlanSearchtable $w365ServicePlanSearchtable -LicenseGroupSearchtable $w365LicenseGroupsSearchTable -Cloud

# Export Array of Hashtables to JSON file and CSV
ConvertTo-Json -InputObject $rawAggregatedRemoteConnections -Depth 100 | Out-File -FilePath "$($FileOutputPath)\Windows365RawAggregatedRemoteConnections.json" -Force
$jsonRawData = Get-Content -Path "$($FileOutputPath)\Windows365RawAggregatedRemoteConnections.json" -Raw | ConvertFrom-Json
$jsonRawData | Export-Csv -Path "$($FileOutputPath)\Windows365RawAggregatedRemoteConnections.csv" -NoTypeInformation

# Export Array of Hashtables to JSON file and CSV
ConvertTo-Json -InputObject $enrichedAggregatedRemoteConnections -Depth 100 | Out-File -FilePath "$($FileOutputPath)\Windows365EnrichedAggregatedRemoteConnections.json" -Force
$jsonEnrichedData = Get-Content -Path "$($FileOutputPath)\Windows365EnrichedAggregatedRemoteConnections.json" -Raw | ConvertFrom-Json
$jsonEnrichedData | Export-Csv -Path "$($FileOutputPath)\Windows365EnrichedAggregatedRemoteConnections.csv" -NoTypeInformation

#-------------------------------------------------------------------------#
# FILTERING Enriched Aggregated Remote Connection Report for Windows365  #
#-------------------------------------------------------------------------#
[int]$daysToAdd = -59
$checkDate = (Get-Date).AddDays($daysToAdd)
$dateFormatString = "MM/dd/yyyy hh:mm:ss tt"
#rule 01: No license check for PRD Cloud PCs | Exclude PRD CloudPCs as they will never expire (for now)
$filteredJsonData = $jsonEnrichedData
$filteredJsonData = $filteredJsonData.Where({ ($_.provisioningPolicyName -notlike 'PRD*') })

#rule 02: For never logged on provisioned CPCs (NeverLoggedIn = true) check CreateDate; if older than date limit, email user that license is revoked
#rule 03: For logged on provisioned CPCs (NeverLoggedIn = false) check LastLoggedInDate; if older than date limit, email user that license is revoked
# Filter CloudPCs with (NeverLoggedIn eq $true and CreatedDate lt $checkDate) or (NeverLoggedIn eq $false and LastActiveTime le $checkDate)
$totalFilteredJsonData = $filteredJsonData.Where({ (($_.NeverSignedIn -eq $true) -and ($_.CreatedDate -lt $checkDate) -and ($_.provisioningStatus -eq 'provisioned')) -or (($_.NeverSignedIn -eq $false) -and ($_.LastActiveTime -le $checkDate) -and ($_.pr

# Loop through the filtered data and add the userId to the hashtable
$enrichedTotalFilteredJsonData = $totalFilteredJsonData
$enrichedTotalFilteredJsonData | ForEach-Object {
    $userPrincipalName = $_.UserPrincipalName
    $userId = Get-W365UserId -AccessToken $($graphContext.access_token) -userPrincipalName $userPrincipalName
    $_ | Add-Member -MemberType NoteProperty -Name "UserDirectoryObjectId" -Value $userId -Force # add the userId to the hashtable
}

# Export Array of Hashtables to JSON file
ConvertTo-Json -InputObject $enrichedTotalFilteredJsonData -Depth 100 | Out-File -FilePath "C:\Temp\W365CloudPCsInactiveLast60Days.json" -Force
$finalJsonData = Get-Content -Path "C:\Temp\W365CloudPCsInactiveLast60Days.json" -Raw | ConvertFrom-Json
$finalJsonData | Export-Csv -Path "C:\Temp\W365CloudPCsInactiveLast60Days.csv" -NoTypeInformation -Force
```

Graph API PowerShell script (workflow)

```
###################################
# Script workflow                 #
###################################
# Get the access token
$graphContext = Get-GraphAccessToken -TenantId $tenantId -AppId $ClientId -AppSecret $ClientSecret
# Build the Searchtables
$w365ServicePlanSearchtable = Get-W365ServicePlans -AccessToken $($graphContext.access_token)
$w365ProvisioningPolicyGroupSearchtable = Get-W365ProvisioningPolicyGroups -AccessToken $($graphContext.access_token) -Provisioni
$w365LicenseGroupsSearchtable = Get-W365LicenseGroups -AccessToken $($graphContext.access_token) -LicenseGroupDisplayName "grp-li
$w365ProvisioningPoliciesSearchtable = Get-W365ProvisioningPolicies -AccessToken $($graphContext.access_token) -ProvisioningPolic
$w365CloudPCs = Get-W365CloudPCs -AccessToken $($graphContext.access_token)

#-----------------------------------------------------------------------#
# Retrieve Intune Total Aggregated Remote Connection Report for Windows365  #
#-----------------------------------------------------------------------#
# Get Intune report data for Total Aggregated Remote Connections
$rawAggregatedRemoteConnections = Get-W365AggregatedRemoteConnectionReport -AccessToken $($graphContext.access_token)
$enrichedAggregatedRemoteConnections = Get-W365AggregatedRemoteConnectionReportWithEnrichedData -AccessToken $($graphContext.acce

# Export Array of Hashtables to JSON file and CSV
ConvertTo-Json -InputObject $rawAggregatedRemoteConnections -Depth 100 | Out-File -FilePath "$($FileOutputPath)\Windows365RawAggr
$jsonRawData = Get-Content -Path "$($FileOutputPath)\Windows365RawAggregatedRemoteConnections.json" -Raw | ConvertFrom-Json
$jsonRawData | Export-Csv -Path "$($FileOutputPath)\Windows365RawAggregatedRemoteConnections.csv" -NoTypeInformation

# Export Array of Hashtables to JSON file and CSV
ConvertTo-Json -InputObject $enrichedAggregatedRemoteConnections -Depth 100 | Out-File -FilePath "$($FileOutputPath)\Windows365Er
$jsonEnrichedData = Get-Content -Path "$($FileOutputPath)\Windows365EnrichedAggregatedRemoteConnections.json" -Raw | ConvertFrom-
$jsonEnrichedData | Export-Csv -Path "$($FileOutputPath)\Windows365EnrichedAggregatedRemoteConnections.csv" -NoTypeInformation
```

Graph API PowerShell script (workflow)

```powershell
###################################
# Script workflow                 #
###################################
# Get the access token
$graphContext = Get-GraphAccessToken -TenantId $tenantId -AppId $ClientId -AppSecret $ClientSecret
# Build the Searchtables
$w365ServicePlanSearchtable = Get-W365ServicePlans -AccessToken $($graphContext.access_token)
$w365ProvisioningPolicyGroupSearchtable = Get-W365ProvisioningPolicyGroups -AccessToken $($graphContext.access_token) -Provisioni
$w365LicenseGroupsSearchtable = Get-W365LicenseGroups -AccessToken $($graphContext.access_token) -LicenseGroupDisplayName "grp-li
$w365ProvisioningPoliciesSearchtable = Get-W365ProvisioningPolicies -AccessToken $($graphContext.access_token) -ProvisioningPoli
$w365CloudPCs = Get-W365CloudPCs -AccessToken $($graphContext.access_token)

#-----------------------------------------------------------------------#
# Retrieve Intune Total Aggregated Remote Connection Report for Windows365  #
#-----------------------------------------------------------------------#
# Get Intune report data for Total Aggregated Remote Connections
$rawAggregatedRemoteConnections = Get-W365AggregatedRemoteConnectionReport -AccessToken $($graphContext.access_token)
$enrichedAggregatedRemoteConnections = Get-W365AggregatedRemoteConnectionReportWithEnrichedData -AccessToken $($graphContext.acce

# Export Array of Hashtables to JSON file and CSV
ConvertTo-Json -InputObject $rawAggregatedRemoteConnections -Depth 100 | Out-File -FilePath "$($FileOutputPath)\Windows365RawAggr
$jsonRawData = Get-Content -Path "$($FileOutputPath)\Windows365RawAggregatedRemoteConnections.json" -Raw | ConvertFrom-Json
$jsonRawData | Export-Csv -Path "$($FileOutputPath)\Windows365RawAggregatedRemoteConnections.csv" -NoTypeInformation

# Export Array of Hashtables to JSON file and CSV
ConvertTo-Json -InputObject $enrichedAggregatedRemoteConnections -Depth 100 | Out-File -FilePath "$($FileOutputPath)\Windows365Er
$jsonEnrichedData = Get-Content -Path "$($FileOutputPath)\Windows365EnrichedAggregatedRemoteConnections.json" -Raw | ConvertFrom-
$jsonEnrichedData | Export-Csv -Path "$($FileOutputPath)\Windows365EnrichedAggregatedRemoteConnections.csv" -NoTypeInformation
```

Graph API PowerShell script (workflow)

```
###################################
# Script workflow                 #
###################################
# Get the access token
$graphContext = Get-GraphAccessToken -TenantId $tenantId -AppId $ClientId -AppSecret $ClientSecret
# Build the Searchtables
$w365ServicePlanSearchtable = Get-W365ServicePlans -AccessToken $($graphContext.access_token)
$w365ProvisioningPolicyGroupSearchtable = Get-W365ProvisioningPolicyGroups -AccessToken $($graphContext.access_token) -Provisioni
$w365LicenseGroupsSearchtable = Get-W365LicenseGroups -AccessToken $($graphContext.access_token) -LicenseGroupDisplayName "grp-li
$w365ProvisioningPoliciesSearchtable = Get-W365ProvisioningPolicies -AccessToken $($graphContext.access_token) -ProvisioningPolic
$w365CloudPCs = Get-W365CloudPCs -AccessToken $($graphContext.access_token)

#-----------------------------------------------------------------------#
# Retrieve Intune Total Aggregated Remote Connection Report for Windows365  #
#-----------------------------------------------------------------------#
# Get Intune report data for Total Aggregated Remote Connections
$rawAggregatedRemoteConnections = Get-W365AggregatedRemoteConnectionReport -AccessToken $($graphContext.access_token)
$enrichedAggregatedRemoteConnections = Get-W365AggregatedRemoteConnectionReportWithEnrichedData -AccessToken $($graphContext.acce

# Export Array of Hashtables to JSON file and CSV
ConvertTo-Json -InputObject $rawAggregatedRemoteConnections -Depth 100 | Out-File -FilePath "$($FileOutputPath)\Windows365RawAggr
$jsonRawData = Get-Content -Path "$($FileOutputPath)\Windows365RawAggregatedRemoteConnections.json" -Raw | ConvertFrom-Json
$jsonRawData | Export-Csv -Path "$($FileOutputPath)\Windows365RawAggregatedRemoteConnections.csv" -NoTypeInformation

# Export Array of Hashtables to JSON file and CSV
ConvertTo-Json -InputObject $enrichedAggregatedRemoteConnections -Depth 100 | Out-File -FilePath "$($FileOutputPath)\Windows365Er
$jsonEnrichedData = Get-Content -Path "$($FileOutputPath)\Windows365EnrichedAggregatedRemoteConnections.json" -Raw | ConvertFrom-
$jsonEnrichedData | Export-Csv -Path "$($FileOutputPath)\Windows365EnrichedAggregatedRemoteConnections.csv" -NoTypeInformation
```

Graph API PowerShell script (workflow)

```powershell
#-----------------------------------------------------------------------#
# FILTERING Enriched Aggregated Remote Connection Report for Windows365  #
#-----------------------------------------------------------------------#
[int]$daysToAdd = -59
$checkDate = (Get-Date).AddDays($daysToAdd)
$dateFormatString = "MM/dd/yyyy hh:mm:ss tt"
#rule 01: No license check for PRD Cloud PCs | Exclude PRD CloudPCs as they will never expire (for now)
$filteredJsonData = $jsonEnrichedData
$filteredJsonData = $filteredJsonData.Where({ ($_.provisioningPolicyName -notlike 'PRD*') })

#rule 02: For never logged on provisioned CPCs (NeverLoggedIn = true) check CreateDate; if older than date limit, email user that
#rule 03: For logged on provisioned CPCs (NeverLoggedIn = false) check LastLoggedInDate; if older than date limit, email user tha
# Filter CloudPCs with (NeverLoggedIn eq $true and CreatedDate lt $checkDate) or (NeverLoggedIn eq $false and LastActiveTime le $
$totalFilteredJsonData = $filteredJsonData.Where({ (($_.NeverSignedIn -eq $true) -and ($_.CreatedDate -lt $checkDate) -and ($_.pr

# Loop through the filtered data and add the userId to the hashtable
$enrichedTotalFilteredJsonData = $totalFilteredJsonData
$enrichedTotalFilteredJsonData | ForEach-Object {
    $userPrincipalName = $_.UserPrincipalName
    $userId = Get-W365UserId -AccessToken $($graphContext.access_token) -userPrincipalName $userPrincipalName
    $_ | Add-Member -MemberType NoteProperty -Name "UserDirectoryObjectId" -Value $userId -Force # add the userId to the hashtabl
}

# Export Array of Hashtables to JSON file
ConvertTo-Json -InputObject $enrichedTotalFilteredJsonData -Depth 100 | Out-File -FilePath "C:\Temp\W365CloudPCsInactiveLast60Day
$finalJsonData = Get-Content -Path "C:\Temp\W365CloudPCsInactiveLast60Days.json" -Raw | ConvertFrom-Json
$finalJsonData | Export-Csv -Path "C:\Temp\W365CloudPCsInactiveLast60Days.csv" -NoTypeInformation -Force
```

Graph API PowerShell script (workflow)

```powershell
#-----------------------------------------------------------------------#
# FILTERING Enriched Aggregated Remote Connection Report for Windows365  #
#-----------------------------------------------------------------------#
[int]$daysToAdd = -59
$checkDate = (Get-Date).AddDays($daysToAdd)
$dateFormatString = "MM/dd/yyyy hh:mm:ss tt"
#rule 01: No license check for PRD Cloud PCs | Exclude PRD CloudPCs as they will never expire (for now)
$filteredJsonData = $jsonEnrichedData
$filteredJsonData = $filteredJsonData.Where({ ($_.provisioningPolicyName -notlike 'PRD*') })

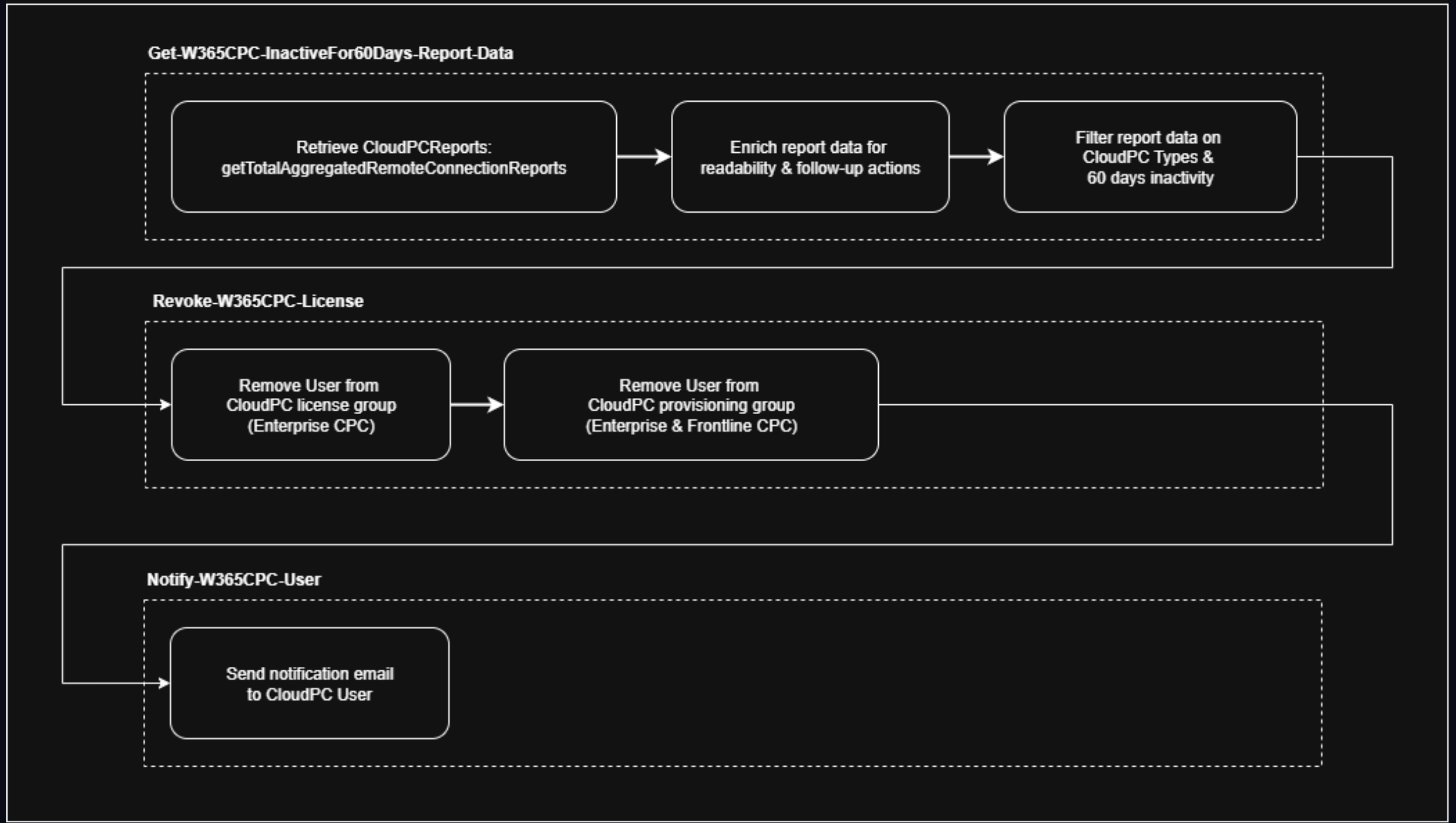
#rule 02: For never logged on provisioned CPCs (NeverLoggedIn = true) check CreateDate; if older than date limit, email user that
#rule 03: For logged on provisioned CPCs (NeverLoggedIn = false) check LastLoggedInDate; if older than date limit, email user tha
# Filter CloudPCs with (NeverLoggedIn eq $true and CreatedDate lt $checkDate) or (NeverLoggedIn eq $false and LastActiveTime le $
$totalFilteredJsonData = $filteredJsonData.Where({ (($_.NeverSignedIn -eq $true) -and ($_.CreatedDate -lt $checkDate) -and ($_.pr

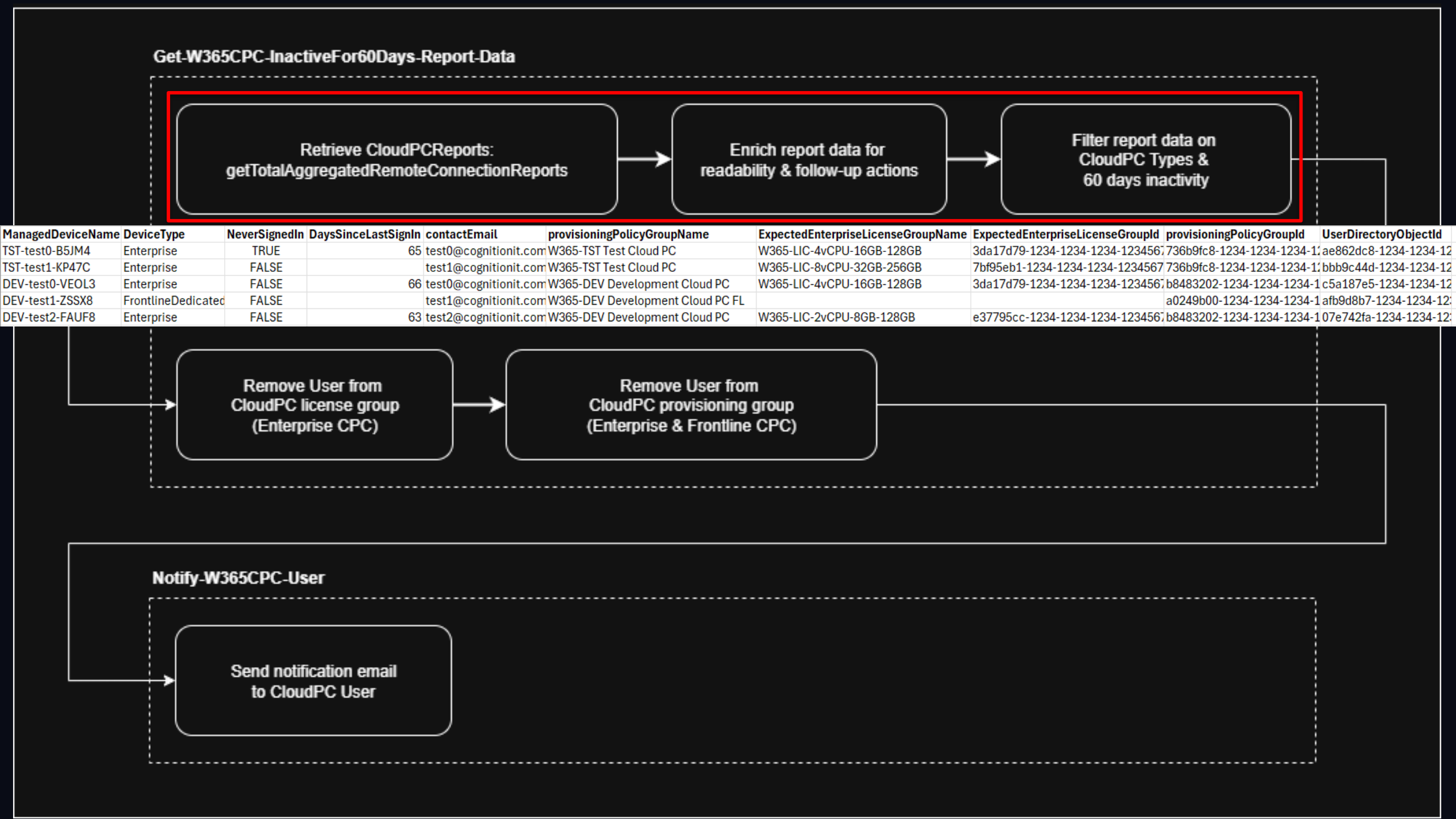
# Loop through the filtered data and add the userId to the hashtable
$enrichedTotalFilteredJsonData = $totalFilteredJsonData
$enrichedTotalFilteredJsonData | ForEach-Object {
    $userPrincipalName = $_.UserPrincipalName
    $userId = Get-W365UserId -AccessToken $($graphContext.access_token) -userPrincipalName $userPrincipalName
    $_ | Add-Member -MemberType NoteProperty -Name "UserDirectoryObjectId" -Value $userId -Force # add the userId to the hashtabl
}

# Export Array of Hashtables to JSON file
ConvertTo-Json -InputObject $enrichedTotalFilteredJsonData -Depth 100 | Out-File -FilePath "C:\Temp\W365CloudPCsInactiveLast60Day
$finalJsonData = Get-Content -Path "C:\Temp\W365CloudPCsInactiveLast60Days.json" -Raw | ConvertFrom-Json
$finalJsonData | Export-Csv -Path "C:\Temp\W365CloudPCsInactiveLast60Days.csv" -NoTypeInformation -Force
```

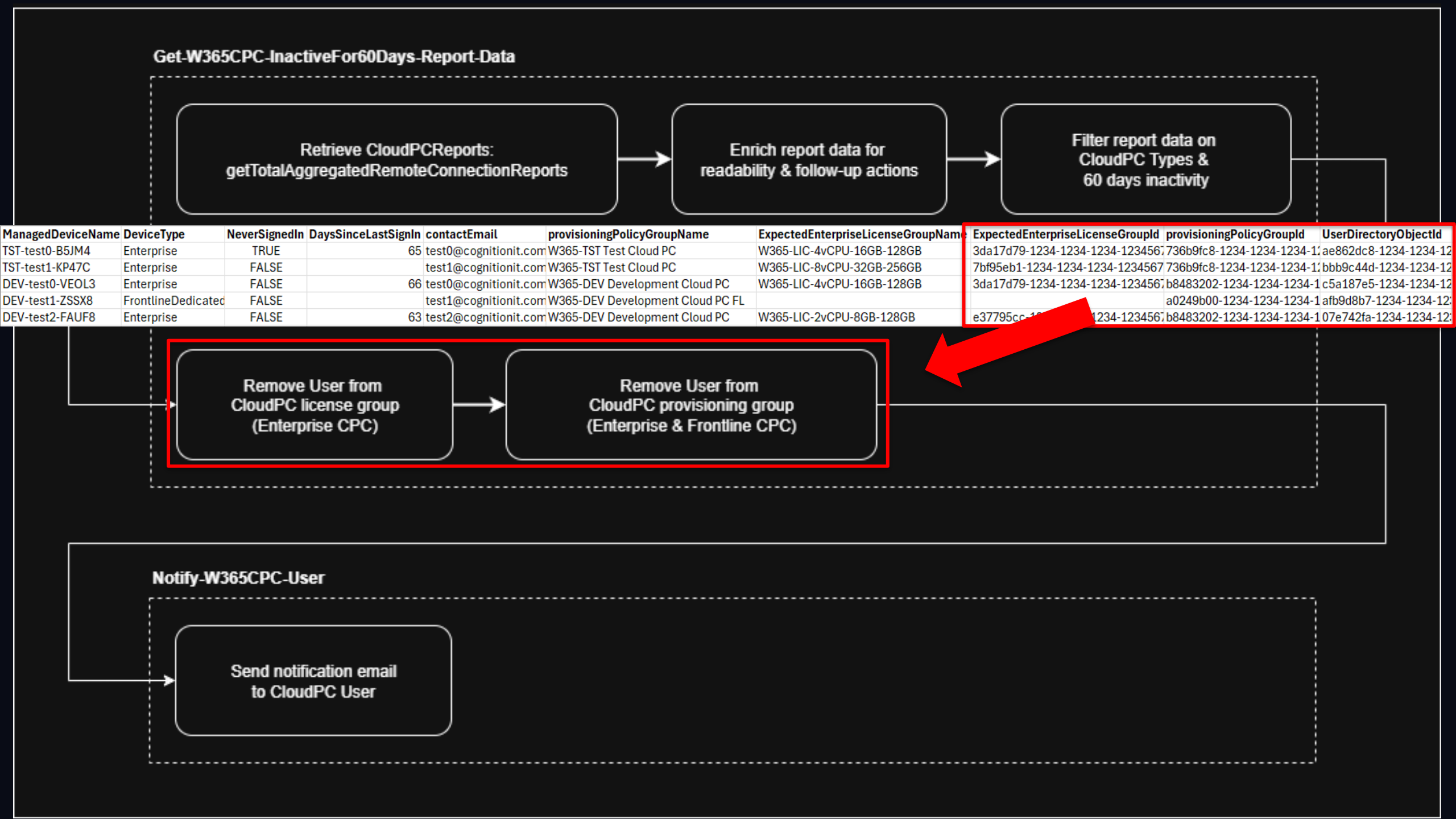
WORKPLACE NINJA SUMMIT

Automation Workflow

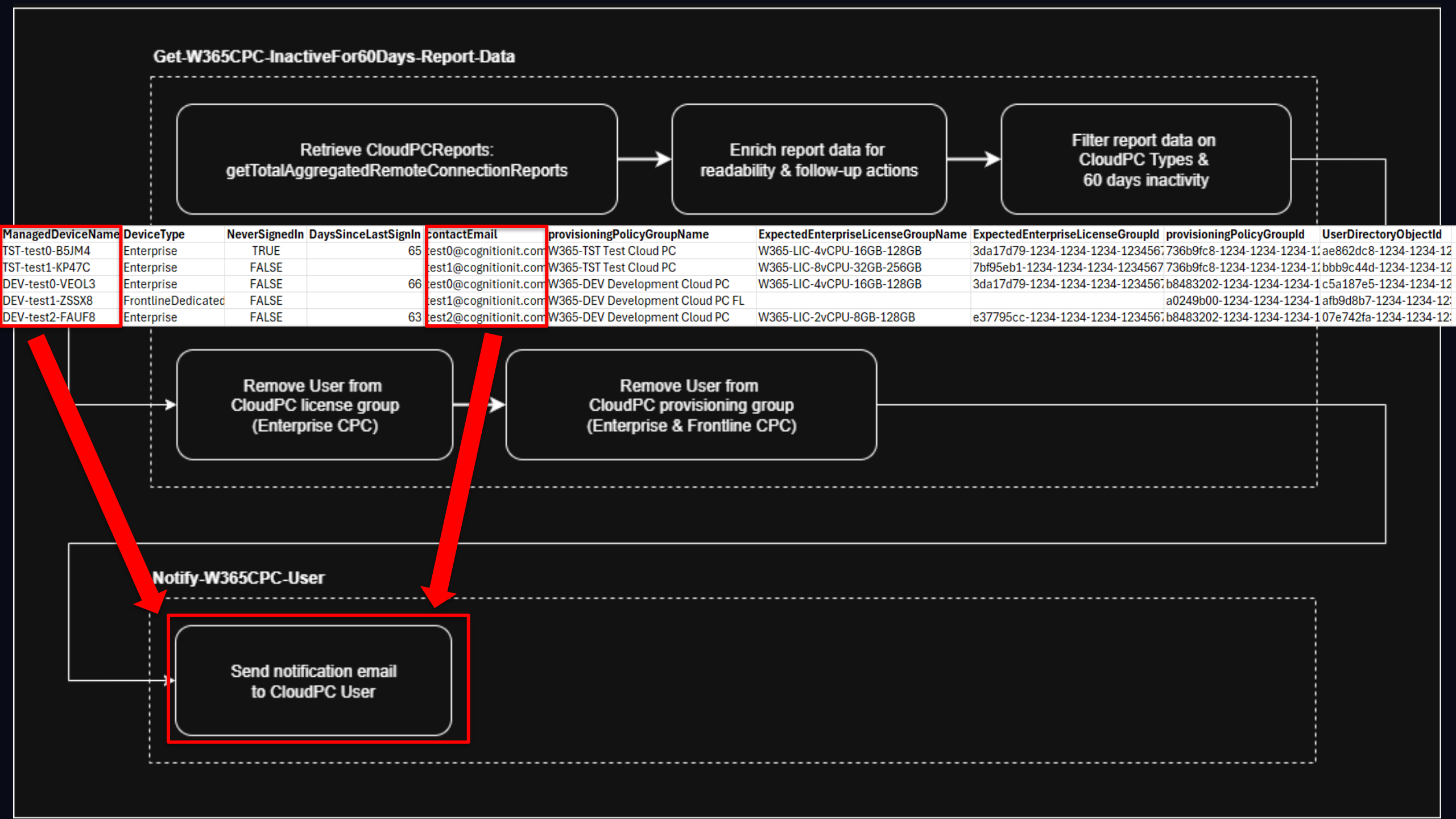

Get-W365CPC-InactiveFor60Days-Report-Data
Retrieve CloudPCReports:
getTotalAggregatedRemoteConnectionReports
Enrich report data for
readability & follow-up actions
Filter report data on
CloudPC Types &
60 days inactivity
Revoke-W365CPC-License
Remove User from
CloudPC license group
(Enterprise CPC)
Remove User from
CloudPC provisioning group
(Enterprise & Frontline CPC)
Notify-W365CPC-User
Send notification email
to CloudPC User



ManagedDeviceName	DeviceType	NeverSignedIn	DaysSinceLastSignIn	contactEmail	provisioningPolicyGroupName	ExpectedEnterpriseLicenseGroupName	ExpectedEnterpriseLicenseGroupId	provisioningPolicyGroupId	UserDirectoryObjectId
TST-test0-B5JM4	Enterprise	TRUE	65	test0@cognitionit.com	W365-TST Test Cloud PC	W365-LIC-4vCPU-16GB-128GB	3da17d79-1234-1234-1234-123456	736b9fc8-1234-1234-1234-1	ae862dc8-1234-1234-12
TST-test1-KP47C	Enterprise	FALSE		test1@cognitionit.com	W365-TST Test Cloud PC	W365-LIC-8vCPU-32GB-256GB	7bf95eb1-1234-1234-1234-1234567	736b9fc8-1234-1234-1234-1	bbb9c44d-1234-1234-12
DEV-test0-VEOL3	Enterprise	FALSE	66	test0@cognitionit.com	W365-DEV Development Cloud PC	W365-LIC-4vCPU-16GB-128GB	3da17d79-1234-1234-1234-123456	b8483202-1234-1234-1234-1	c5a187e5-1234-1234-12
DEV-test1-ZSSX8	FrontlineDedicated	FALSE		test1@cognitionit.com	W365-DEV Development Cloud PC FL			a0249b00-1234-1234-1234-1	afb9d8b7-1234-1234-12
DEV-test2-FAUF8	Enterprise	FALSE	63	test2@cognitionit.com	W365-DEV Development Cloud PC	W365-LIC-2vCPU-8GB-128GB	e37795cc-1234-1234-1234-123456	b8483202-1234-1234-1234-1	07e742fa-1234-1234-12



ManagedDeviceName	DeviceType	NeverSignedIn	DaysSinceLastSignIn	contactEmail	provisioningPolicyGroupName	ExpectedEnterpriseLicenseGroupName	ExpectedEnterpriseLicenseGroupId	provisioningPolicyGroupId	UserDirectoryObjectId
TST-test0-B5JM4	Enterprise	TRUE	65	test0@cognitionit.com	W365-TST Test Cloud PC	W365-LIC-4vCPU-16GB-128GB	3da17d79-1234-1234-1234-123456	736b9fc8-1234-1234-1234-1	ae862dc8-1234-1234-12
TST-test1-KP47C	Enterprise	FALSE		test1@cognitionit.com	W365-TST Test Cloud PC	W365-LIC-8vCPU-32GB-256GB	7bf95eb1-1234-1234-1234-1234567	736b9fc8-1234-1234-1234-1	bbb9c44d-1234-1234-12
DEV-test0-VEOL3	Enterprise	FALSE	66	test0@cognitionit.com	W365-DEV Development Cloud PC	W365-LIC-4vCPU-16GB-128GB	3da17d79-1234-1234-1234-123456	b8483202-1234-1234-1234-1	c5a187e5-1234-1234-12
DEV-test1-ZSSX8	FrontlineDedicated	FALSE		test1@cognitionit.com	W365-DEV Development Cloud PC FL			a0249b00-1234-1234-1234-1	afb9d8b7-1234-1234-12
DEV-test2-FAUF8	Enterprise	FALSE	63	test2@cognitionit.com	W365-DEV Development Cloud PC	W365-LIC-2vCPU-8GB-128GB	e37795cc-1 1234-123456	b8483202-1234-1234-1234-1	07e742fa-1234-1234-12



ManagedDeviceName	DeviceType	NeverSignedIn	DaysSinceLastSignIn	contactEmail	provisioningPolicyGroupName	ExpectedEnterpriseLicenseGroupName	ExpectedEnterpriseLicenseGroupId	provisioningPolicyGroupId	UserDirectoryObjectId
TST-test0-B5JM4	Enterprise	TRUE	65	test0@cognitionit.com	W365-TST Test Cloud PC	W365-LIC-4vCPU-16GB-128GB	3da17d79-1234-1234-1234-123456	736b9fc8-1234-1234-1234-1	ae862dc8-1234-1234-12
TST-test1-KP47C	Enterprise	FALSE		test1@cognitionit.com	W365-TST Test Cloud PC	W365-LIC-8vCPU-32GB-256GB	7bf95eb1-1234-1234-1234-1234567	736b9fc8-1234-1234-1234-1	bbb9c44d-1234-1234-12
DEV-test0-VEOL3	Enterprise	FALSE	66	test0@cognitionit.com	W365-DEV Development Cloud PC	W365-LIC-4vCPU-16GB-128GB	3da17d79-1234-1234-1234-123456	b8483202-1234-1234-1234-1	c5a187e5-1234-1234-12
DEV-test1-ZSSX8	FrontlineDedicated	FALSE		test1@cognitionit.com	W365-DEV Development Cloud PC FL			a0249b00-1234-1234-1234-1	afb9d8b7-1234-1234-12
DEV-test2-FAUF8	Enterprise	FALSE	63	test2@cognitionit.com	W365-DEV Development Cloud PC	W365-LIC-2vCPU-8GB-128GB	e37795cc-1234-1234-1234-123456	b8483202-1234-1234-1234-1	07e742fa-1234-1234-12

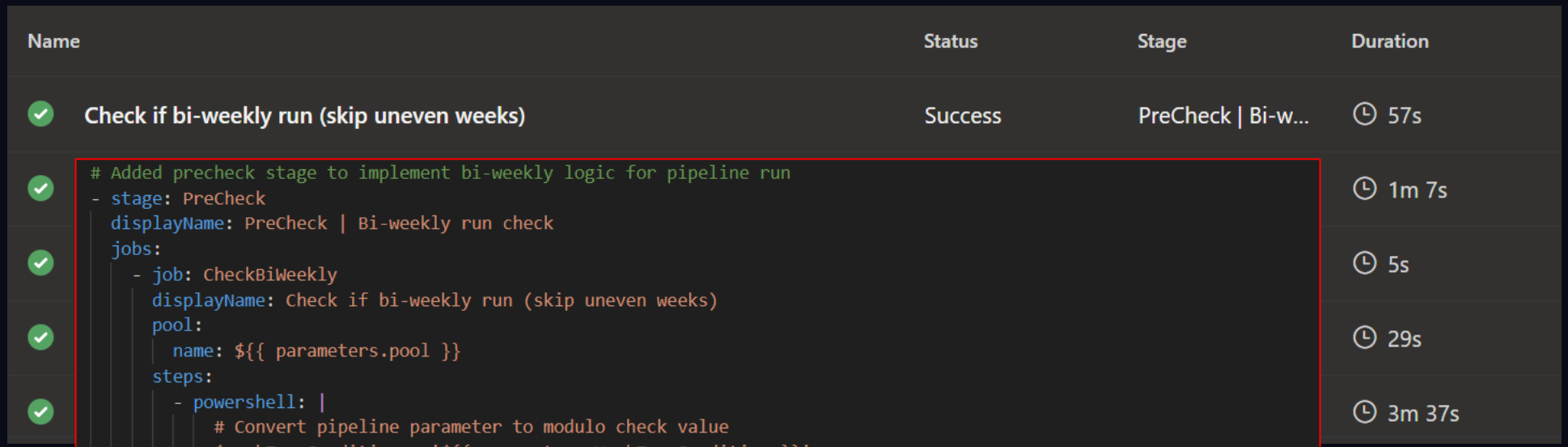
Bi-weekly schedule hack

Name	Status	Stage	Duration
Check if bi-weekly run (skip uneven weeks)	Success	PreCheck | Bi-w...	49s
A-1 | Generate W365 unused license report and send Teams channel notific...	Skipped	Stage A | Filter ...	
B-1 | Approval for License Revocation	Skipped	Stage B | Revok...	
B-2 | Teams notification for Approval	Skipped	Stage B | Revok...	
B-3 | Revoke W365 Licenses	Skipped	Stage B | Revok...	

Bi-weekly schedule hack

Name	Status	Stage	Duration
Check if bi-weekly run (skip uneven weeks)	Success	PreCheck | Bi-w...	57s
A-1 | Generate W365 unused license report and send Teams channel notific...	Success	Stage A | Filter ...	1m 7s
B-1 | Approval for License Revocation	Success	Stage B | Revok...	5s
B-2 | Teams notification for Approval	Success	Stage B | Revok...	29s
B-3 | Revoke W365 Licenses	Success	Stage B | Revok...	3m 37s

Bi-weekly schedule hack



Name	Status	Stage	Duration
Check if bi-weekly run (skip uneven weeks)	Success	PreCheck | Bi-w...	57s
			1m 7s
			5s
			29s
			3m 37s

```yaml
# Added precheck stage to implement bi-weekly logic for pipeline run
- stage: PreCheck
  displayName: PreCheck | Bi-weekly run check
  jobs:
    - job: CheckBiWeekly
      displayName: Check if bi-weekly run (skip uneven weeks)
      pool:
        name: ${{ parameters.pool }}
      steps:
        - powershell: |
            # Convert pipeline parameter to modulo check value
            $weekTypeCondition = '${{ parameters.WeekTypeCondition }}'
            $expectedModulo = if ($weekTypeCondition -eq 'even') { 0 } else { 1 }

            if ($weekNumber % 2 -eq $expectedModulo) {
                Write-Verbose "$weekTypeCondition week - running pipeline as per bi-weekly schedule."
                Write-Output "##vso[task.setvariable variable=SkipPipeline;isOutput=true]false"
            } else {
                Write-Output "##[warning]${weekTypeCondition} week - skipping pipeline run as per bi-weekly schedule."
                Write-Output "##vso[task.setvariable variable=SkipPipeline;isOutput=true]true"
            }
          name: SetSkipPipelineVariable
          displayName: Set SkipPipeline variable
```

```yaml
- stage: A
  displayName: Stage A | Filter W365 License Usage Intune Report
  # check for bi-weekly run
  dependsOn: PreCheck
  condition: eq(dependencies.PreCheck.outputs['CheckBiWeekly.SetSkipPipelineVariable.SkipPipeline'], 'false')
  jobs:
```

Pipeline approval

Name	Status	Stage	Duration
Check if bi-weekly run (skip uneven weeks)	Success	PreCheck | Bi-w...	57s
A-1 | Generate W365 unused license report and send Teams channel notific...	Success	Stage A | Filter ...	1m 7s
B-1 | Approval for License Revocation	Success	Stage B | Revok...	5s
			29s
			3m 37s

```yaml
jobs:
  - job: ApprovalCheck
    displayName: B-1 | Approval for License Revocation
    pool: server # ManualValidation task can only run on server, not self-hosted agent pool
    steps:
      - task: ManualValidation@0
        displayName: 'Request License Revocation Approval'
        timeoutInMinutes: 4320 # 3 days
        inputs:
          notifyUsers: |
            $(ApproverEmails) # From variable group
          instructions: |
            ## W365 License Revocation Approval Required

            This pipeline will revoke Windows 365 licenses that have been identified as underutilized.
            Please review the attached license report before approving:
            **Build Number**: $(Build.BuildNumber)
            **Licenses to be revoked**: $(W365LicensesToRevokeCount)

            Note: a list of all unused licenses is available in the W365LicenseUsageReports artifact (and in the Teams notification).
          onTimeout: 'reject'
```



Manual Validation

Resumed

Job

B-1 | Approval for License Revocation

Completed
Resumed • 1 Sept

Instructions

W365 License Revocation Approval Required

This pipeline will revoke Windows 365 licenses that have been identified as underutilized.
Please review the attached license report before approving:
Build Number: W365_License_Usage_Report_1491288_20260831.1
Licenses to be revoked: 27

Note: a list of all unused licenses is available in the W365LicenseUsageReports artifact (and in the Teams notification).

Teams notifications

WORKPLACE NINJA SUMMIT

W365CloudPCsInactiveLast60Days

Pipeline: main.W365LicenseUsageReport
Build: 1491288 | Job: A-1 | Generate W365 unused license report and send Teams channel notification (Succeeded)

27 unused W365 license(s) found.

W365CloudPCsInactiveLast60Days

Pipeline: main.W365LicenseUsageReport
Build: 1491288 | Job: B-2 | Teams notification for Approval (Succeeded)

An approval is required to revoke **27** unused W365 license(s).

ManagedDeviceName	LastActiveTime	NeverSignedIn
DEV-	06/23/2026 16:21:53	Signed In
DEV-		Never

Review & Approve Pipeline build
(to start the license(s) removal action)

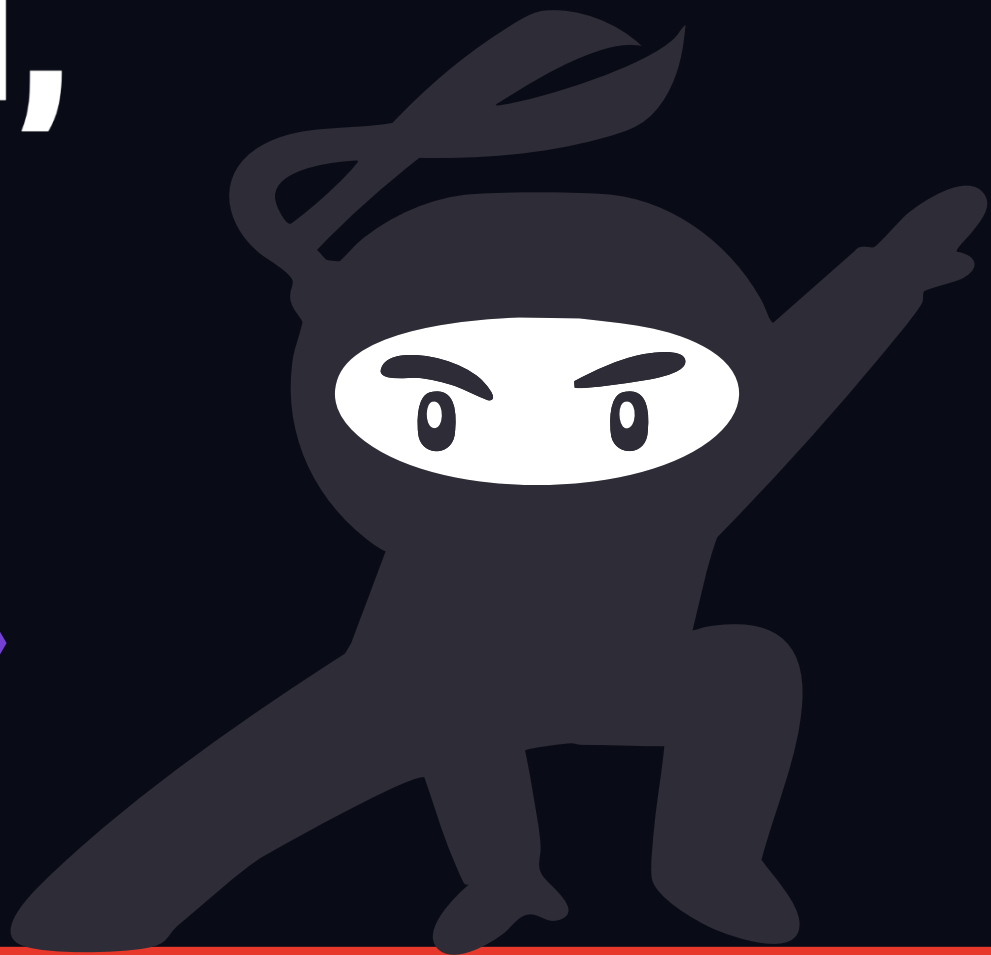
W365CloudPCsLicenseRevocationMailingList

Pipeline: main.W365LicenseUsageReport
Build: 1474709 | Job: B-3 | Revoke W365 Licenses (Succeeded)

64 W365 license(s) revoked | Success Count **Success Count: 80** | Error Count **Error Count: 0**

Contact email	CloudPC

Thank You, Ninjas!



github.com/cloud-devops-ninja/myPresentations/tree/main/GraphAPI_W365LicenseManagement



cloud-devops-ninja / myPresentations

Type / to search

Code | Issues | Pull requests | Actions | Projects | Wiki | Security | Insights | Settings

Files

main

Go to file t

- Alyx
- ExpertsLive
- GraphAPI_StarterKit
  - Code
  - Postman
  - Slides
- GraphAPI_W365LicenseManagement
  - Code
    - GraphAPI_W365LicenseReport.ps1
  - Slides
    - AVDTF_GraphAPI_W365_License_Management_v1.pdf

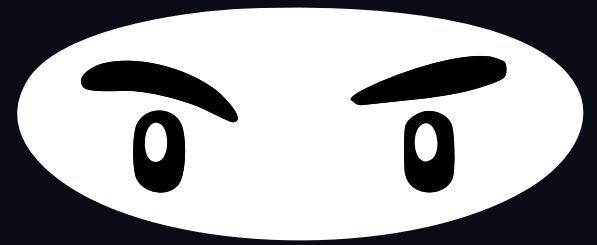
myPresentations / GraphAPI_W365LicenseManagement /

cloud-devops-ninja AVDTechFest update 0129856 · 4 minutes ago History

Name	Last commit message	Last commit date
..		
Code	AVDTechFest update	4 minutes ago
Slides	AVDTechFest update	4 minutes ago

See You Next Year!

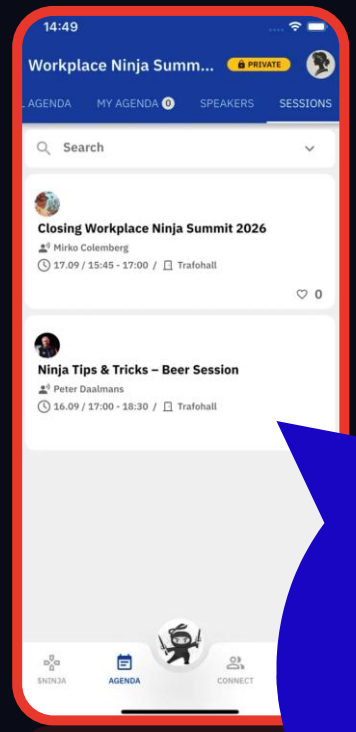
SUMMIT
SUMMIT
SUMMIT
SUMMIT
SUMMIT



Download the app and never miss any updates

GET IT ON Google Play Store

Download on the App Store



We appreciate your feedback!